CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação

Prof. Marco André Abud Kappel

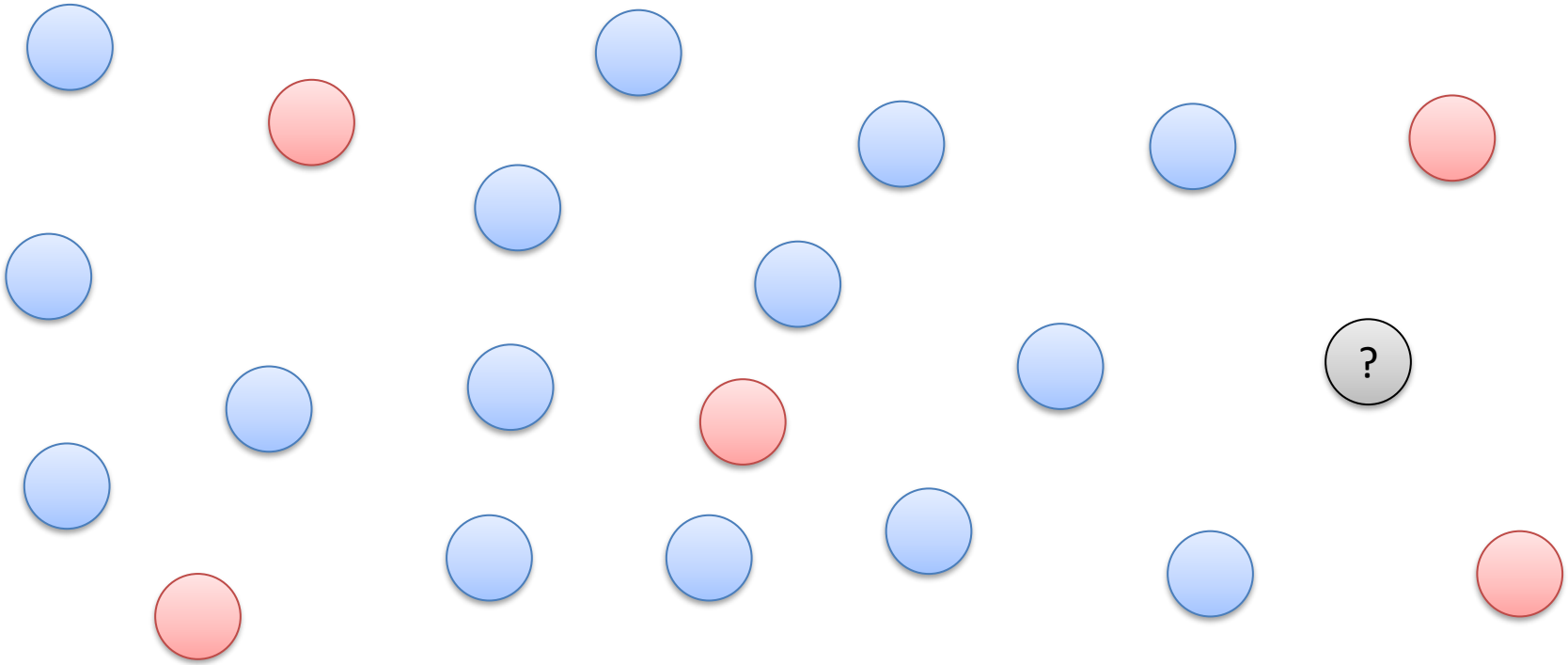
Aula 4 – K-Nearest Neighbours

Classificador Básico

- **Introdução**
  - Existe um **critério mínimo** para saber se um algoritmo de classificação está sendo bem sucedido.
  - O critério é: ter um resultado melhor que o **classificador básico**!
  - O algoritmo do classificador básico é simples:
    - A classe do novo registro será a **mais frequente** na base de treinamento.
  - Esse algoritmo também é chamado de “Classificação por classe majoritária”

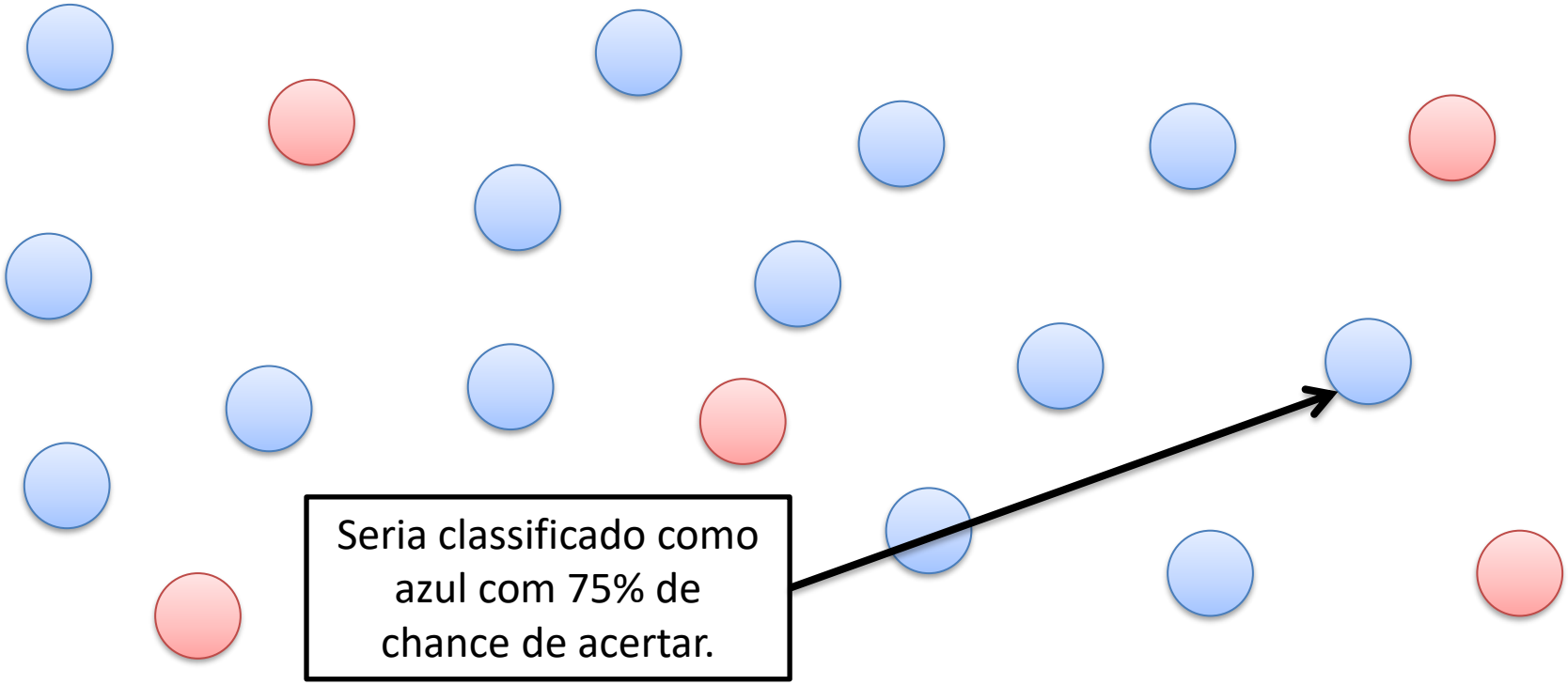
Classificador Básico

- **Exemplo**
  - Considerando a base abaixo, como seria classificado o novo registro?



Classificador Básico

- **Exemplo**
  - Considerando a base abaixo, como seria classificado o novo registro?



Classificador Básico

- **Exemplo**
  - O próximo passo seria conferir a **taxa de acerto** deste critério na base de testes.
  - Este resultado seria o **mínimo necessário** para que qualquer método de aprendizado de máquina **justifique** sua complexidade.

- **Construindo o script**

  – Primeiramente, o script de pré-processamento deve ser executado, selecionando quais as transformações precisam ser feitas na base.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\PythonWorkspace\2025.1\aula4

...tual\2025-1_Técnicas de Aprendizado de Máquina (reformulação)\aula 3\Programas\pre-processamento.py

pre-processamento.py × credit_data.csv ×

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 31 18:32:59 2025

@author: marco
"""

import pandas as pd
base = pd.read_csv('credit_data.csv')
resumo = base.describe()


# ==========================================================================
#                        Tratando valores inválidos
# ==========================================================================

# Verifica idades inválidas
base.loc[base['Age'] < 0]

# Apagar a coluna
# base.drop('Age', axis=1, inplace=True)

# Apagar apenas os registros com problema
# base.drop(base[base['Age'] < 0].index, inplace=True)

# Preencher os valores com a média
idade_media = base['Age'][base['Age'] > 0].mean().round()
base.loc[base['Age'] < 0, 'Age'] = idade_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, …
checking_maioria	str	6	little
classe	DataFrame	(600, 1)	Column names: Risk
classe_teste	DataFrame	(150, 1)	Column names: Risk
classe_treinamento	DataFrame	(450, 1)	Column names: Risk
cols_classe	list	1	['Risk']
cols_previsores	Index	(18,)	Index: 18 entries, Age …

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024,
15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]:
   ...: """
   ...: Created on Mon Mar 31 18:32:59 2025
   ...:
   ...: @author: marco
   ...: """
   ...:
   ...: import pandas as pd
   ...: base = pd.read_csv('credit_data.csv')
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 3, Col 36 UTF-8 CRLF RW Mem 75%

- **Construindo o script**
  - Para construir o classificador básico, crie um novo arquivo chamado "classificador_basico.py".
  - Nele, insira o seguinte código:

```
################## Classificação por classe majoritária ##################

# Resultado mínimo - ajuste pela classe majoritária
contagem = classe_treinamento['Risk'].value_counts()
classe_majoritaria = contagem.idxmax()

previsoes = pd.Series([classe_majoritaria]).repeat(classe_teste.size)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)
```

  - Execute e veja a taxa de acerto do algoritmo:

Nome	Tipo	Tamanho	
acuracia_teste	float	1	0.4666666666666667
matriz_teste	Array of int64	(2, 2)	[[70 0] [80 0]]

Precisamos registrar este resultado para futuras comparações!

- **Registrando os resultados**
  - Cada método será executado diversas vezes, com diversas configurações.
  - É necessário registrar todos os resultados em um arquivo como o mostrado abaixo, para futura análise.

Base de Dados	
Descrição	
Total de registros:	
Total de características:	
Total de colunas de classes:	

Resultados				
Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão

- **Registrando os resultados**
  - O arquivo atualizado com os resultados para esta base fica assim:

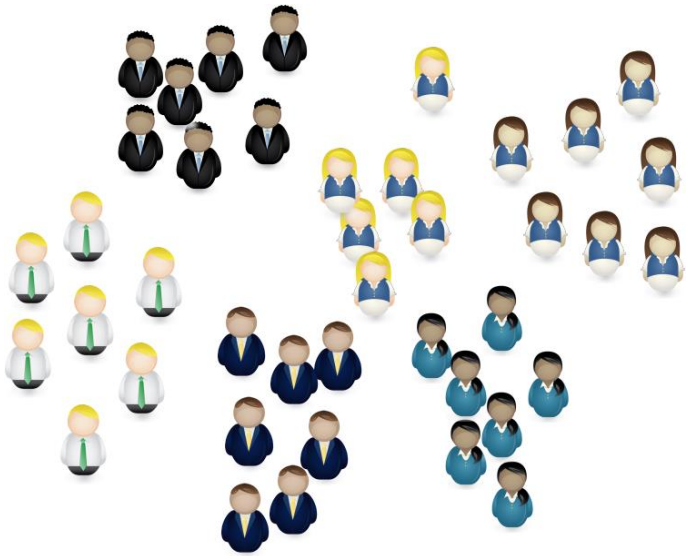
Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados				
Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão
Classe Majoritária	-	-	0.4667	70 0 80 0

K-Nearest Neighbours

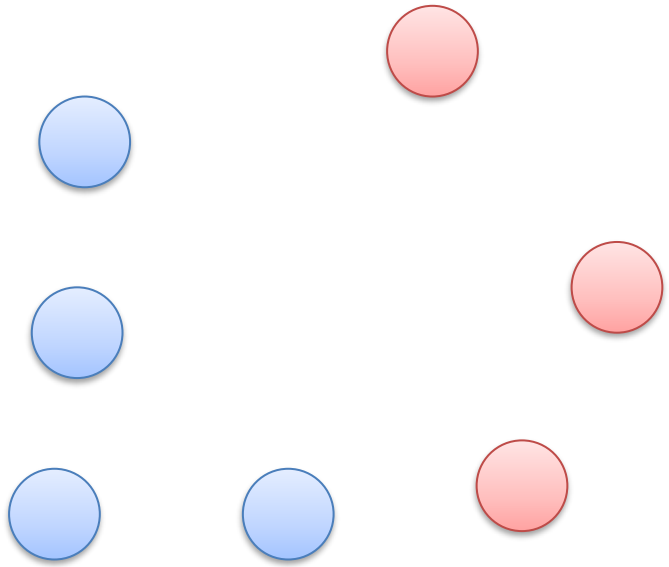
K-Nearest Neighbours

- **Introdução**
  - Uma das ideias mais **simples** para classificação.
  - Baseada em **instâncias**.
  - Envolve o cálculo da **distância** entre diferentes registros.



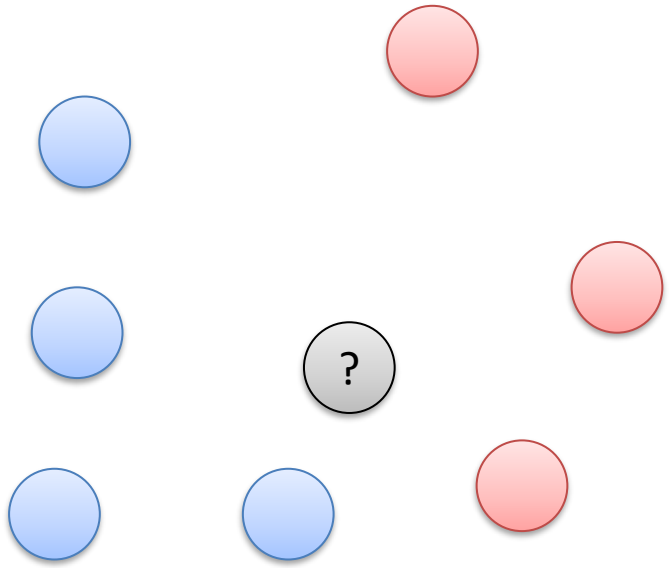
K-Nearest Neighbours

- **Descrição da técnica**
  - A classificação de um **novo registro** é baseada nos K vizinhos mais próximos a ele.



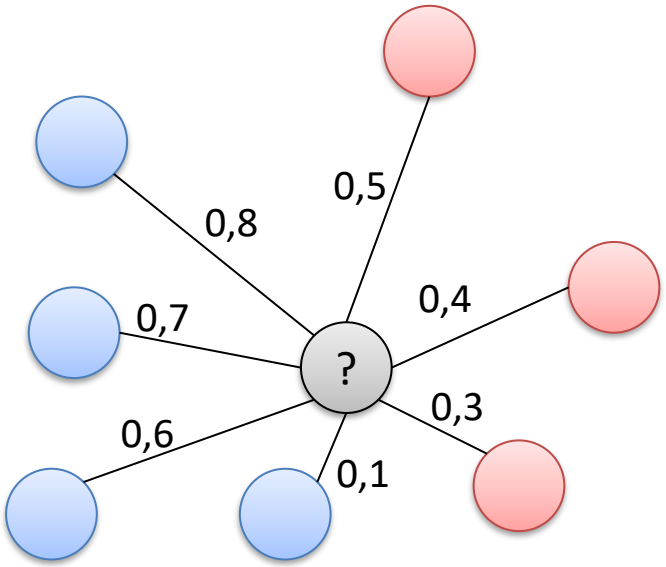
K-Nearest Neighbours

- **Descrição da técnica**
  - Temos um conjunto de **dados históricos** no sistema.
  - O objetivo é classificar o novo dado entre **azul** ou **vermelho**.



K-Nearest Neighbours

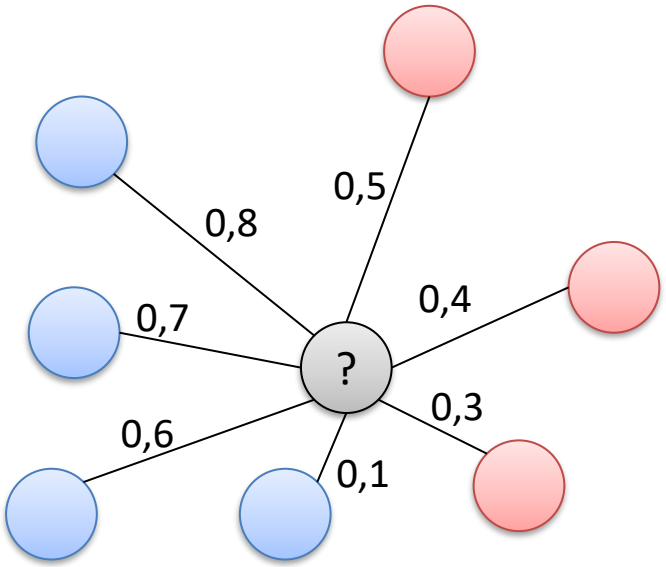
- **Descrição da técnica**
  - Primeiro, será necessário calcular a **distância** entre o novo registro a todos os outros.



Veremos que existem diversas formas diferentes de calcular distâncias.

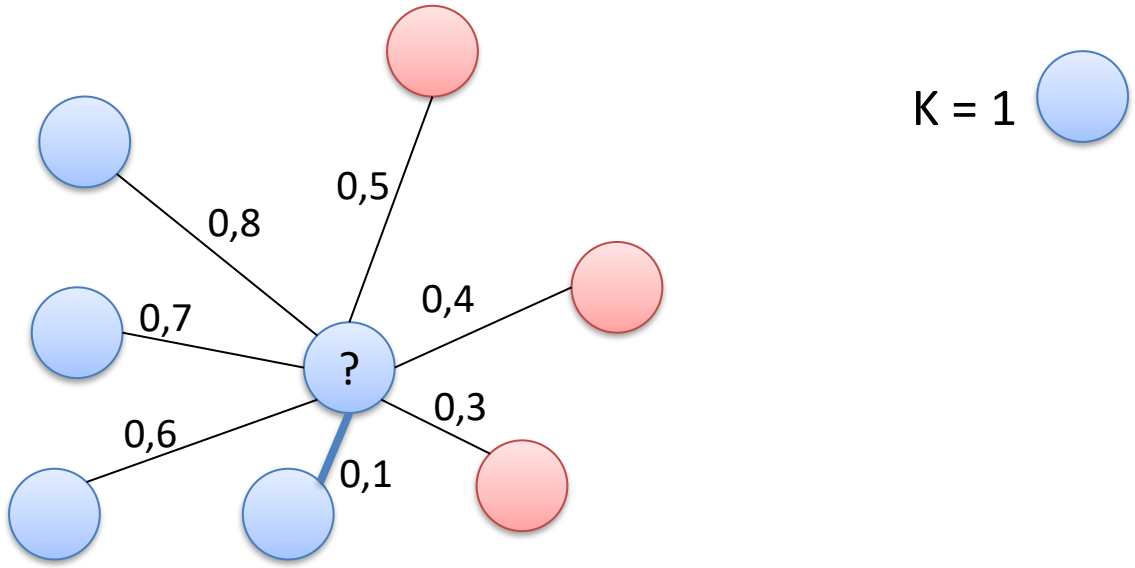
K-Nearest Neighbours

- **Descrição da técnica**
  - O valor escolhido para o parâmetro **K** irá definir **quantos vizinhos** serão utilizados na classificação do novo registro.



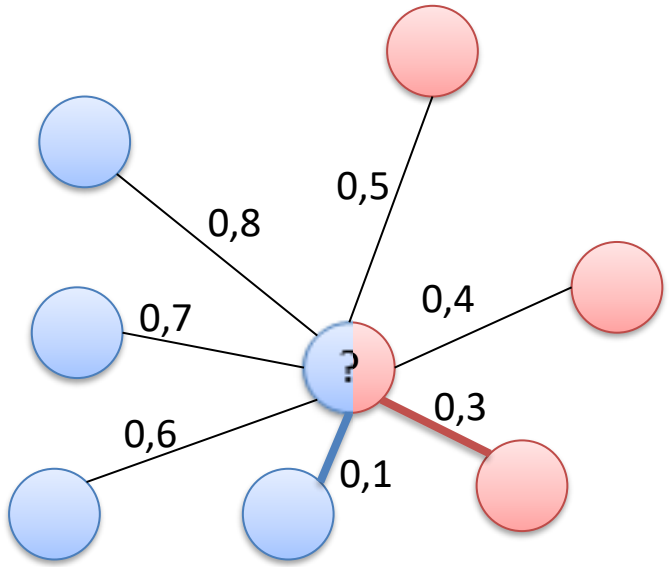
K-Nearest Neighbours

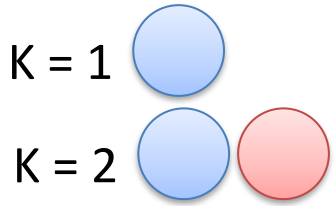
- **Descrição da técnica**
  - Se for utilizado o valor de K = 1, apenas o **vizinho mais próximo** será usado na classificação.



K-Nearest Neighbours

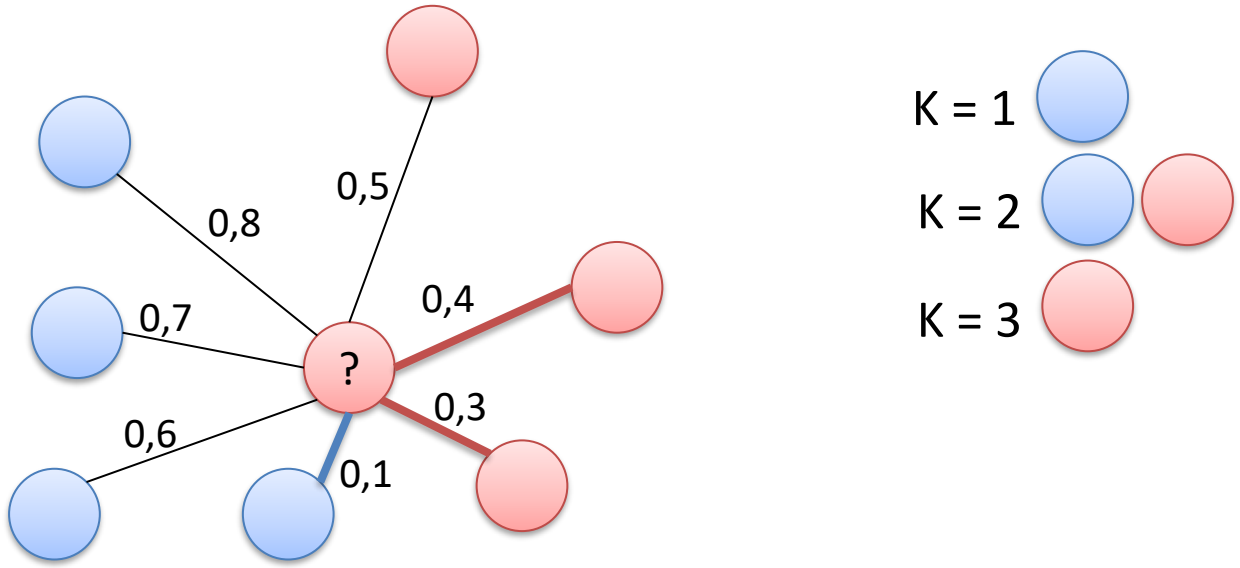
- **Descrição da técnica**
  - Se for utilizado o valor de K = 2, as **duas menores distâncias** serão utilizadas. Neste caso, haverá um empate.





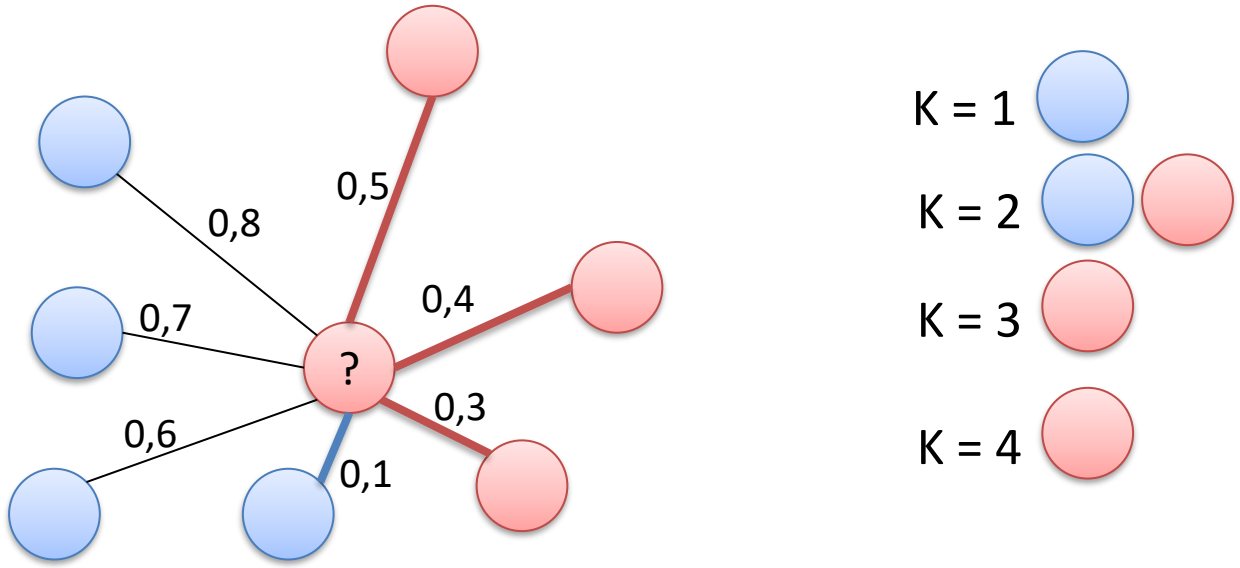
K-Nearest Neighbours

- **Descrição da técnica**
  - Se for utilizado o valor de K = 3, as **três menores distâncias** serão utilizadas.



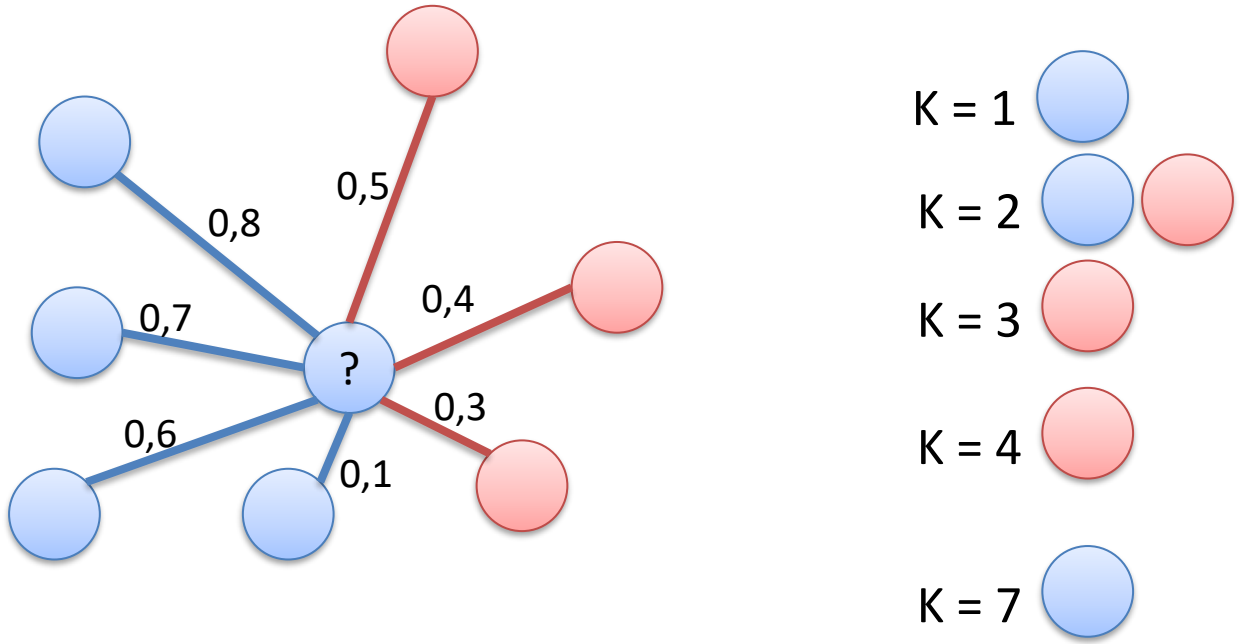
K-Nearest Neighbours

- **Descrição da técnica**
  - Se for utilizado o valor de K = 4, as **quatro menores distâncias** serão utilizadas.



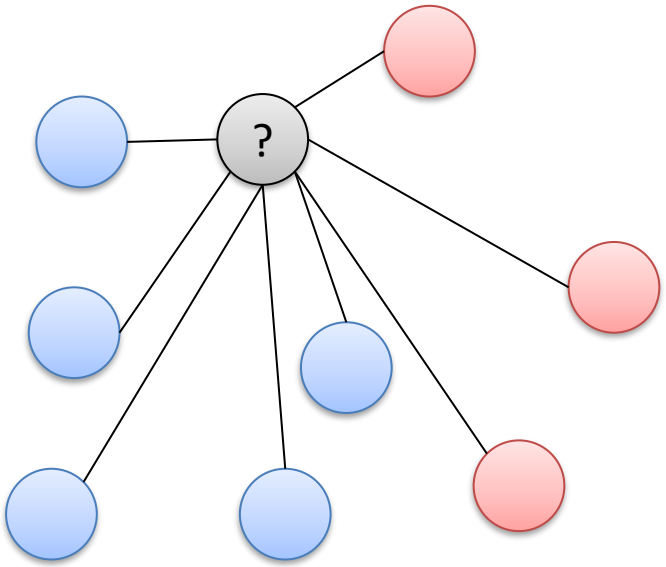
K-Nearest Neighbours

- **Descrição da técnica**
  - No caso extremo, em que usamos K = 7, o algoritmo irá apenas classificar o novo registro com a **classe mais frequente** na base.



K-Nearest Neighbours

- **Descrição da técnica**
  - Para cada **novo registro**, será necessário calcular sua distância para todos os anteriores.



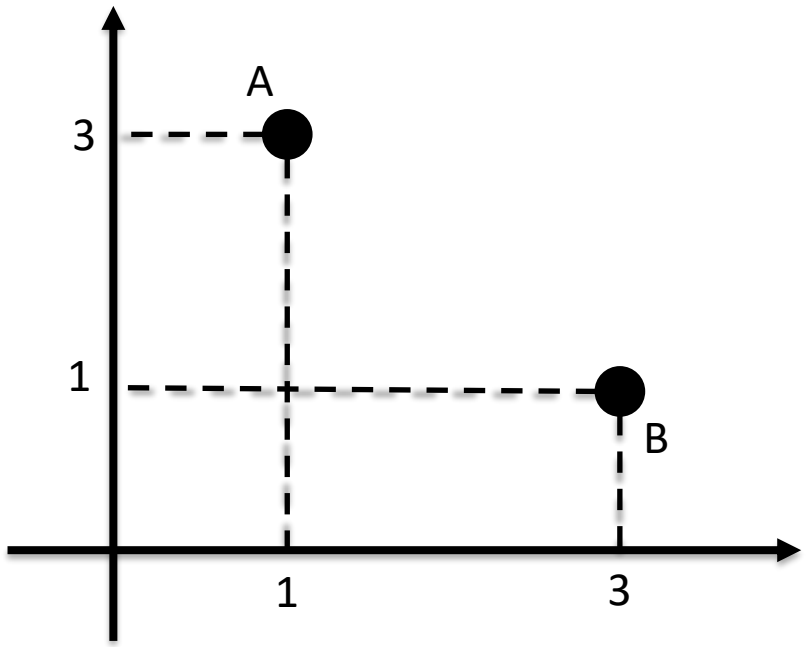
K-Nearest Neighbours

- **Cálculo das distâncias**
  - Existem diferentes **fórmulas** que representam o cálculo da distância.
  - Algumas podem funcionar melhor que outras em **casos diferentes**.
  - Todas representam, de alguma maneira, o grau de **similaridade** entre os registros da base de dados.
  - Alguns exemplos:
    - **Euclidiana**
    - Manhattan
    - Minkowski
    - Correlação de Pearson
    - Cosseno

Veremos essa com maior profundidade, por ser a mais simples e usada.

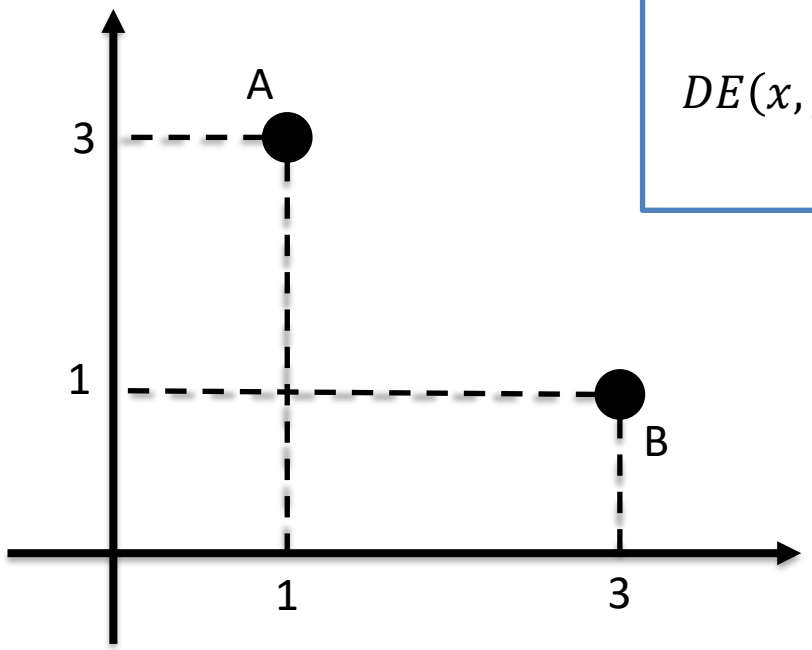
K-Nearest Neighbours

- **Distância Euclidiana**
  - Representa a distância entre dois pontos em um **plano cartesiano**.



K-Nearest Neighbours

- **Distância Euclidiana**
  - Representa a distância entre dois pontos em um **plano cartesiano**.

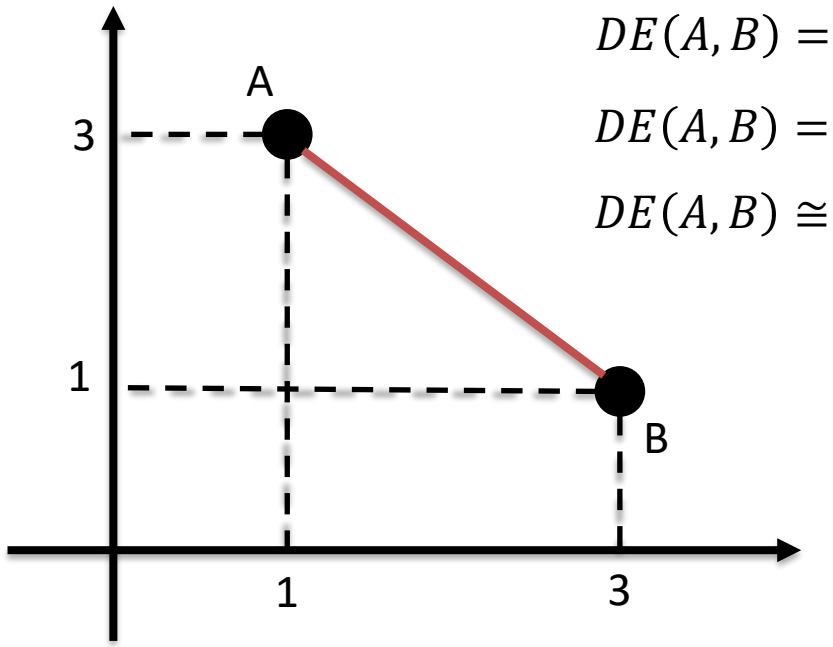


$$DE(x, y) = \sqrt{\sum_{i}^{p} (x_i - y_i)^2}$$

K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p}(x_i - y_i)^2}$$

- **Distância Euclidiana**
  - Representa a distância entre dois pontos em um **plano cartesiano**.



$$DE(A,B) = \sqrt{(3-1)^2 + (1-3)^2}$$

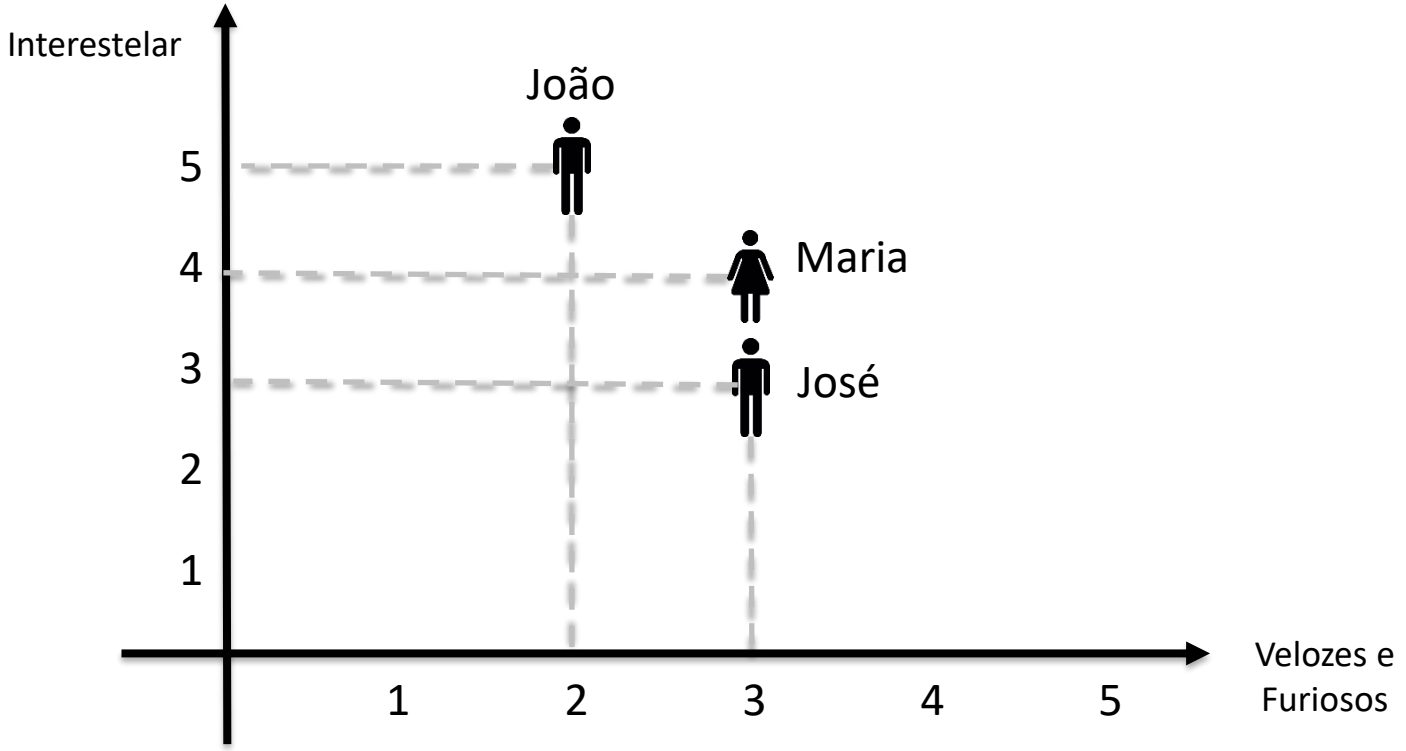
$$DE(A,B) = \sqrt{4+4}$$

$$DE(A,B) \cong \mathbf{2,83}$$

K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p} (x_i - y_i)^2}$$

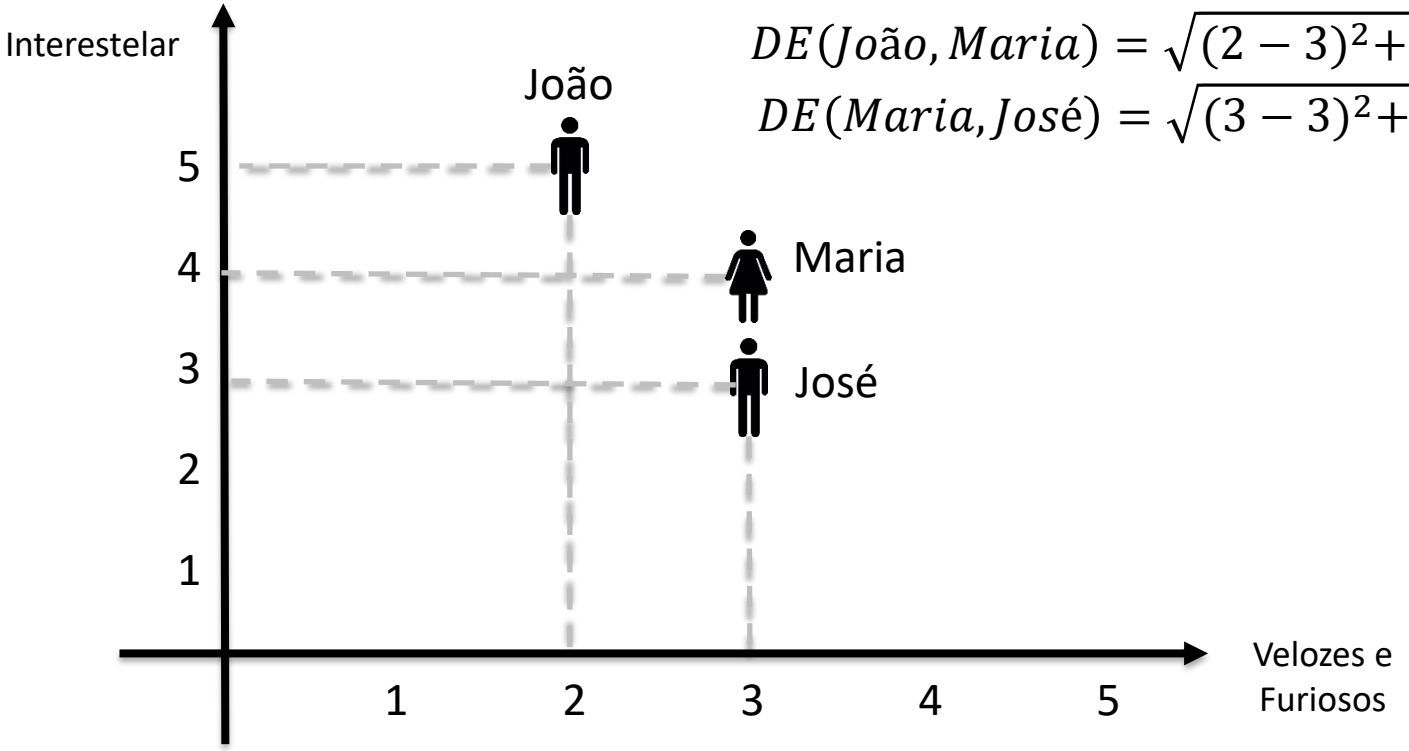
- **Distância Euclidiana**
  - Exemplo: Sistemas de Recomendação (Filtragem Colaborativa)



K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p}(x_i - y_i)^2}$$

- **Distância Euclidiana**
  - Exemplo: Sistemas de Recomendação (Filtragem Colaborativa)



$$DE(João, Maria) = \sqrt{(2-3)^2 + (5-4)^2} = 1{,}41$$

$$DE(Maria, José) = \sqrt{(3-3)^2 + (4-3)^2} = \mathbf{1}$$

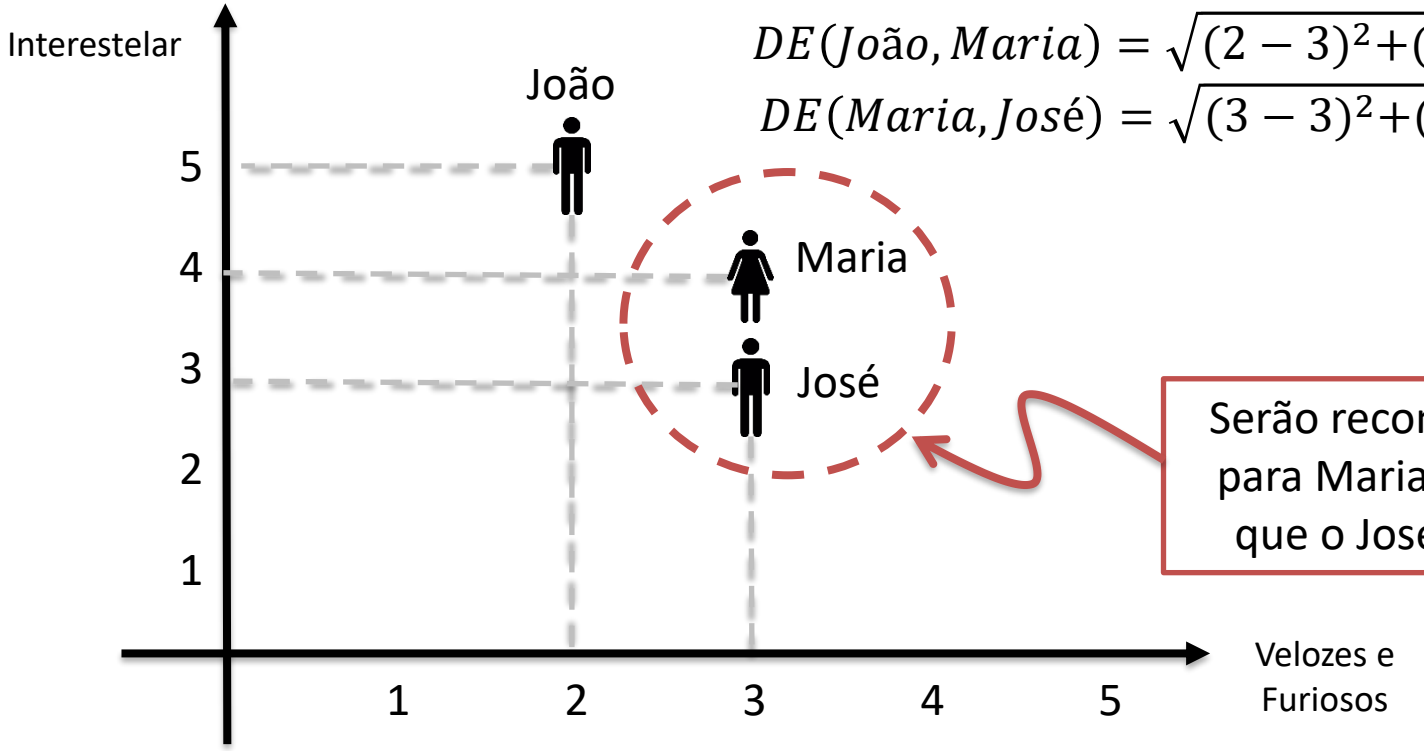
K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p}(x_i - y_i)^2}$$

- **Distância Euclidiana**
  - Exemplo: Sistemas de Recomendação (Filtragem Colaborativa)

$$DE(João, Maria) = \sqrt{(2-3)^2 + (5-4)^2} = 1{,}41$$

$$DE(Maria, José) = \sqrt{(3-3)^2 + (4-3)^2} = \mathbf{1}$$

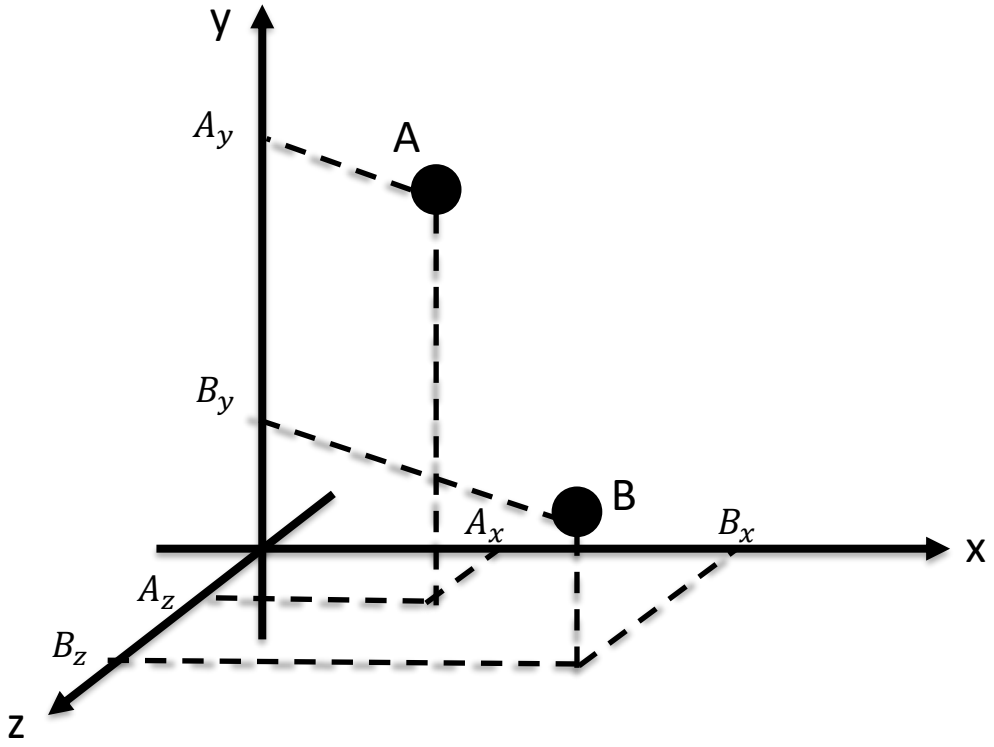


Serão recomendados para Maria os filmes que o José gostou.

K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p}(x_i - y_i)^2}$$

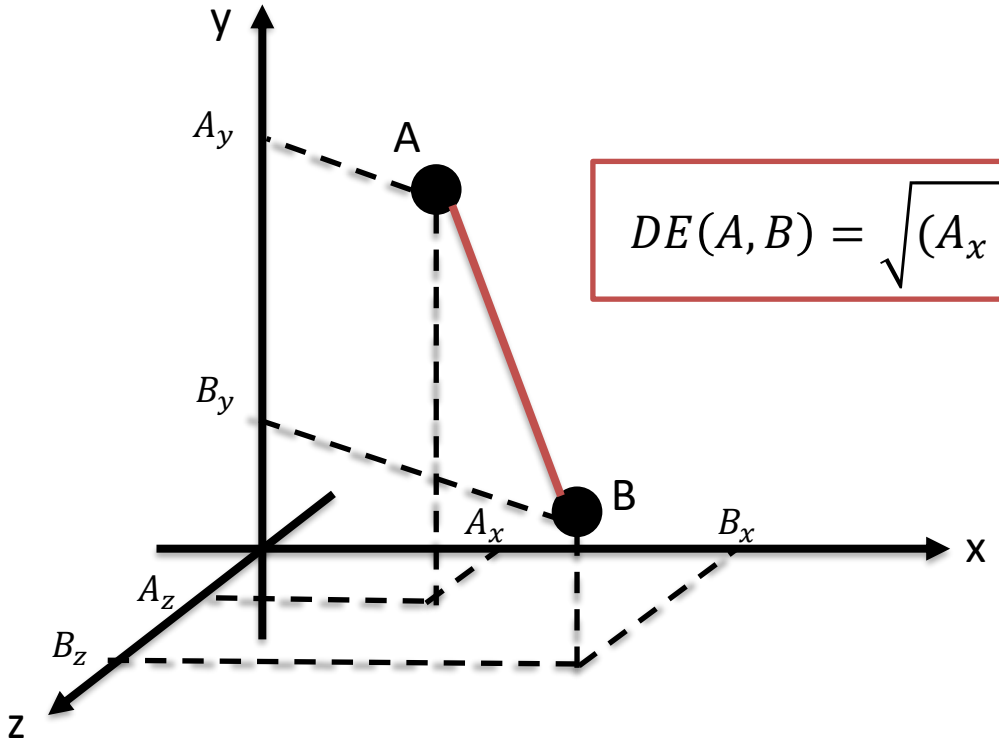
- **Distância Euclidiana**
  - Pode ser aplicada em qualquer número de dimensões.



K-Nearest Neighbours

$$DE(x,y) = \sqrt{\sum_{i}^{p}(x_i - y_i)^2}$$

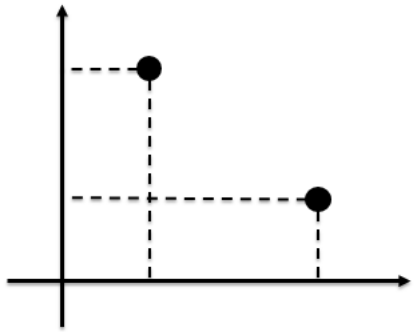
- **Distância Euclidiana**
  - Pode ser aplicada em qualquer número de dimensões.



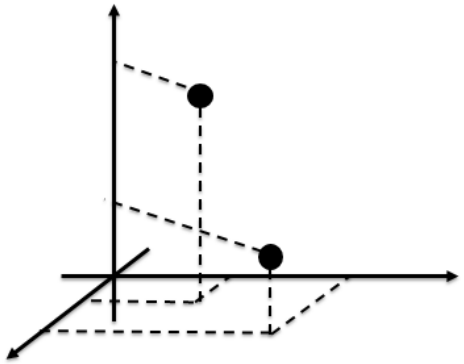
$$DE(A,B) = \sqrt{(A_x - B_x)^2 + (A_y - B_y)^2 + (A_z - B_z)^2}$$

K-Nearest Neighbours

- **Aplicação em bases reais**
  - Cada **dimensão** corresponde a uma **característica** na base de dados.
  - Uma base de dados real pode possuir muitas características, ou seja, são problemas **multidimensionais**.
  - Apesar de não ser possível visualizar **graficamente** todas as dimensões, as fórmulas de distância se aplicam da **mesma forma**.



2 dimensões



3 dimensões

?

N dimensões

K-Nearest Neighbours

- **Aplicação em bases reais**
  - Por exemplo, considere a **base de dados** abaixo:

História de Crédito	Dívida	Garantias	Renda anual	Risco
Média	Baixa	Nenhuma	15k a 35k	Alto
Média	Alta	Nenhuma	15k a 35k	Moderado
Média	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	> 35.000	Baixo
Ruim	Alta	Nenhuma	< 15.000	Alto

K-Nearest Neighbours

- **Aplicação em bases reais**
  - Depois da etapa de **pré-processamento**, a base ficaria assim:

História de Crédito	Dívida	Garantias	Renda anual	Risco
2	3	1	2	Alto
2	2	1	2	Moderado
2	2	1	3	Baixo
1	2	1	3	Baixo
3	2	1	1	Alto

K-Nearest Neighbours

- **Aplicação em bases reais**
  - Suponha que um **novo registro** é inserido.
  - Queremos **classificá-lo** como de risco baixo, moderado ou alto.

História de Crédito	Dívida	Garantias	Renda anual	Risco
2	3	1	2	Alto
2	2	1	2	Moderado
2	2	1	3	Baixo
1	2	1	3	Baixo
3	2	1	1	Alto
1	1	1	3	?

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$DE(x,y) = \sqrt{\sum_{i}^{p} (x_i - y_i)^2}$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$DE(novo, 4) = \sqrt{(1-1)^2+(1-2)^2+(1-1)^2+(3-3)^2}$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$DE(novo, 4) = \sqrt{(1-1)^2+(1-2)^2+(1-1)^2+(3-3)^2}$$

$$= \sqrt{0+1+0+0}$$

$$= 1$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$\boxed{DE(novo, 4) = 1}$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$DE(novo, 2) = \sqrt{(1-2)^2+(1-2)^2+(1-1)^2+(3-2)^2}$$

$$\boxed{DE(novo, 4) = 1}$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$DE(novo, 2) = \sqrt{(1-2)^2+(1-2)^2+(1-1)^2+(3-2)^2}$$

$$= \sqrt{1+1+0+1}$$

$$= 1{,}73$$

$$\boxed{DE(novo, 4) = 1}$$

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	?

$$\boxed{DE(novo, 4) = 1}$$

$$\boxed{DE(novo, 2) = 1{,}73}$$

  - Dentre esses dois, o novo registro está mais próximo de quem?

- **Aplicação em bases reais**
  - Primeiramente, é necessário calcular a distância do novo registro para todos os outros.
  - Vejamos dois exemplos do cálculo da distância:

	História de Crédito	Dívida	Garantias	Renda anual	Risco
1	2	3	1	2	Alto
2	2	2	1	2	Moderado
3	2	2	1	3	Baixo
4	1	2	1	3	Baixo
5	3	2	1	1	Alto
6	1	1	1	3	Baixo

$$\boxed{DE(novo, 4) = 1}$$

$$\boxed{DE(novo, 2) = 1{,}73}$$

  - Dentre esses dois, o novo registro está mais próximo do **registro 4**.
  - Assim, ele seria classificado como de **baixo risco**.

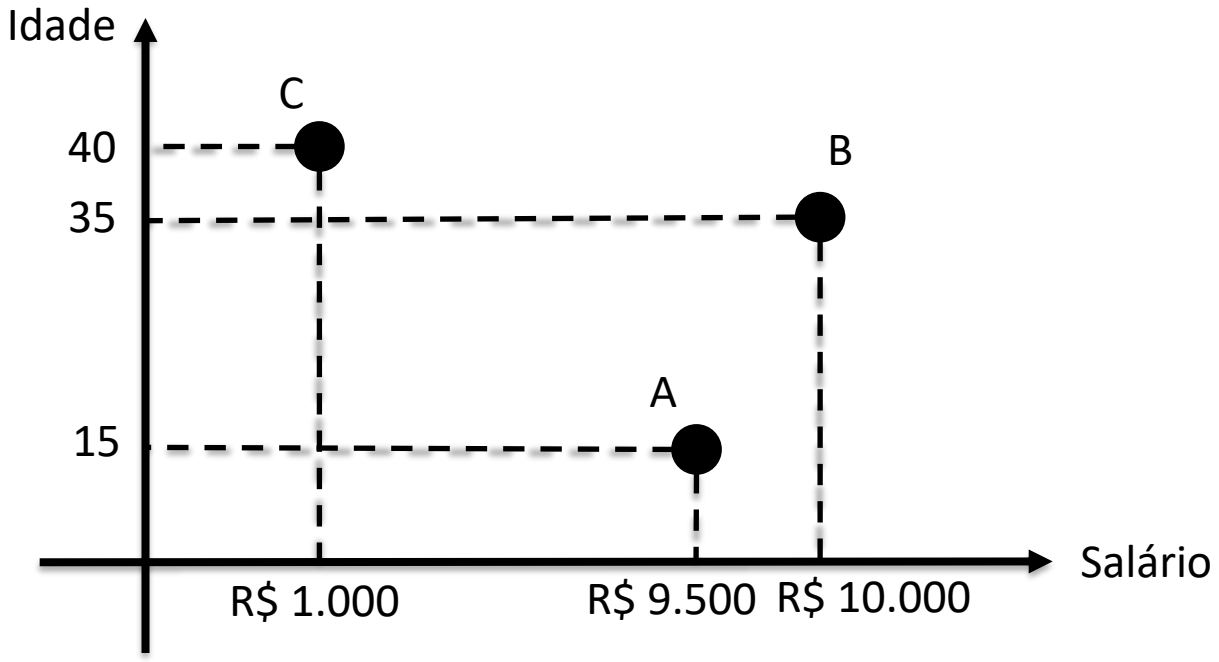
K-Nearest Neighbours

- **Aplicação em bases reais**
  - Para uma aplicação bem sucedida do KNN, dois cuidados devem ser tomados:
    - **Os valores precisam ser numéricos**: não será possível aplicar fórmulas de distância se os valores forem simbólicos.
    - **As variáveis devem estar na mesma escala**: escalas diferentes irão causar pesos diferentes para cada atributo no cálculo da distância.

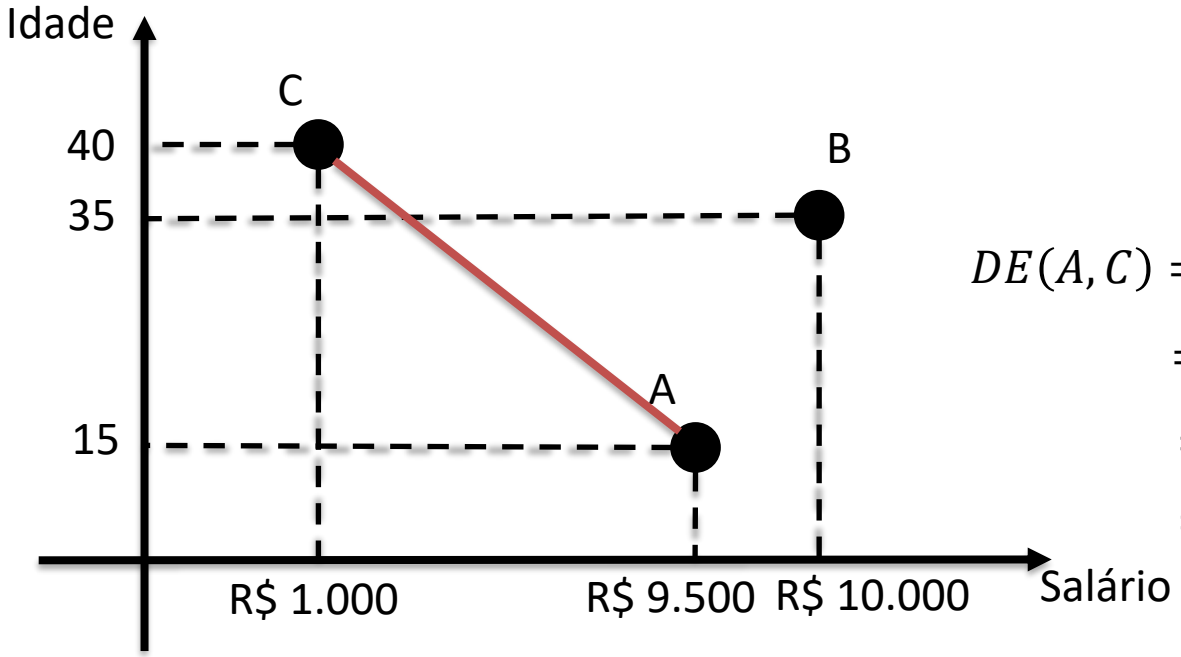
K-Nearest Neighbours

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000



- **Aplicação em bases reais**
  - Exemplo:

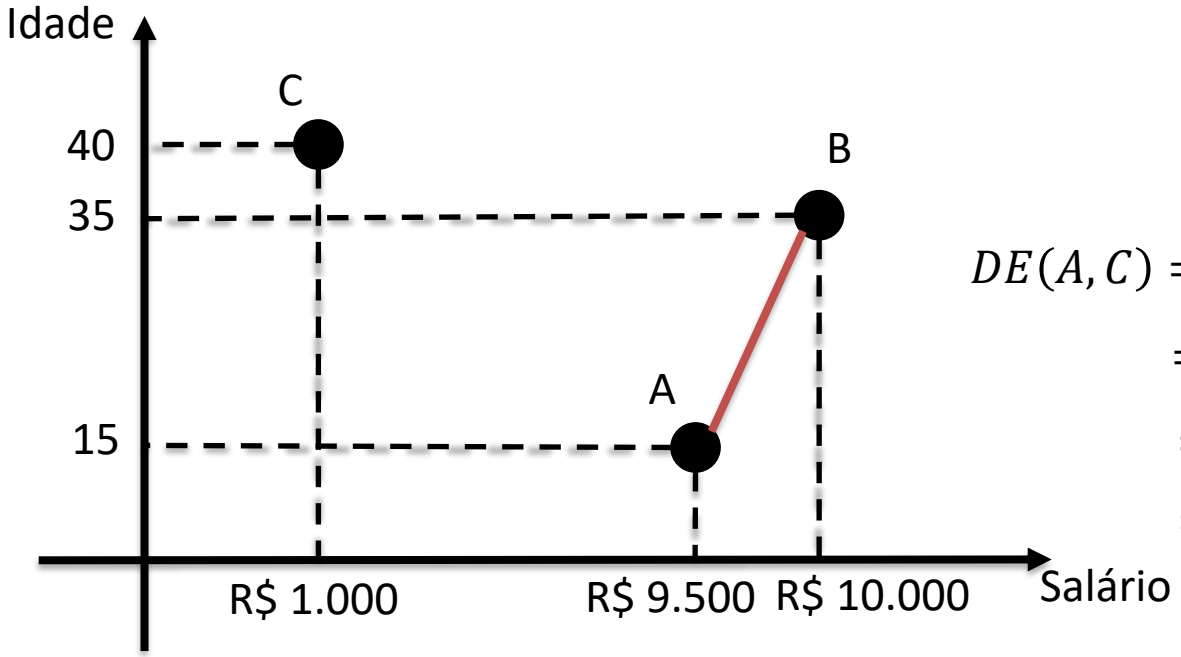


	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000

$$DE(A,C) = \sqrt{(15-40)^2+(9500-1000)^2}$$
$$= \sqrt{(25)^2+(8500)^2}$$
$$= \sqrt{625+72250000}$$
$$= 8500{,}04$$

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000

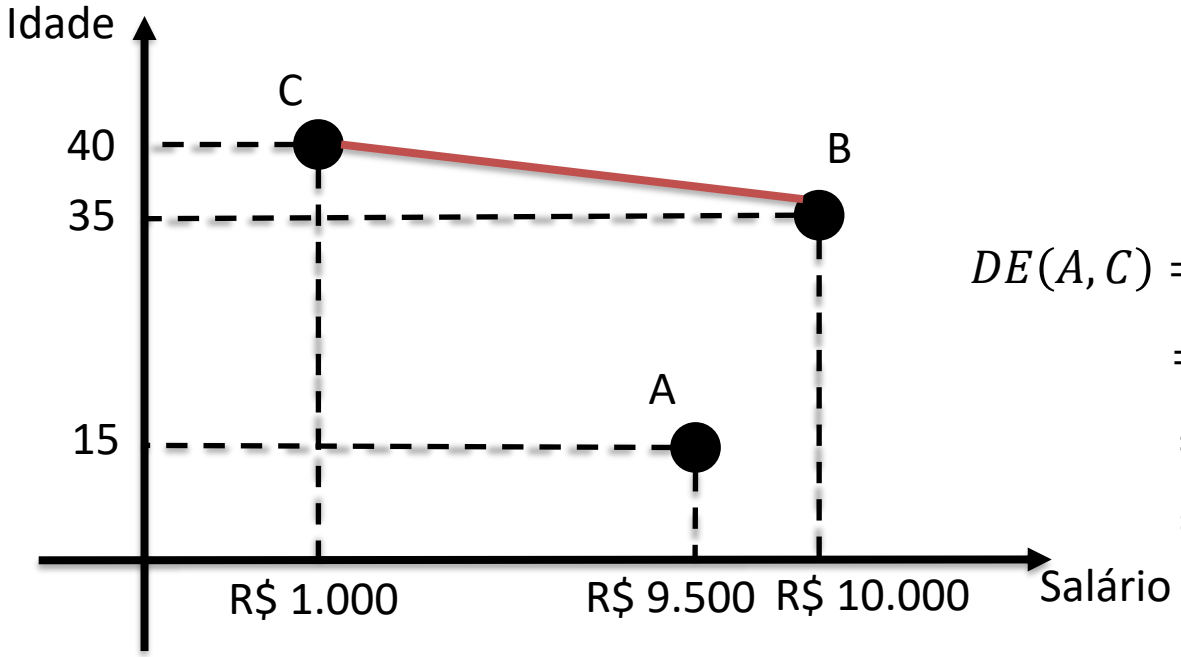


$$DE(A,C) = \sqrt{(15-40)^2+(9500-1000)^2}$$
$$= \sqrt{(25)^2+(8500)^2}$$
$$= \sqrt{625+72250000}$$
$$= 8500{,}04$$

$$DE(A,B) = \sqrt{(15-35)^2+(9500-10000)^2}$$
$$= \sqrt{(20)^2+(-500)^2}$$
$$= \sqrt{400+250000}$$
$$= 500{,}40$$

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000



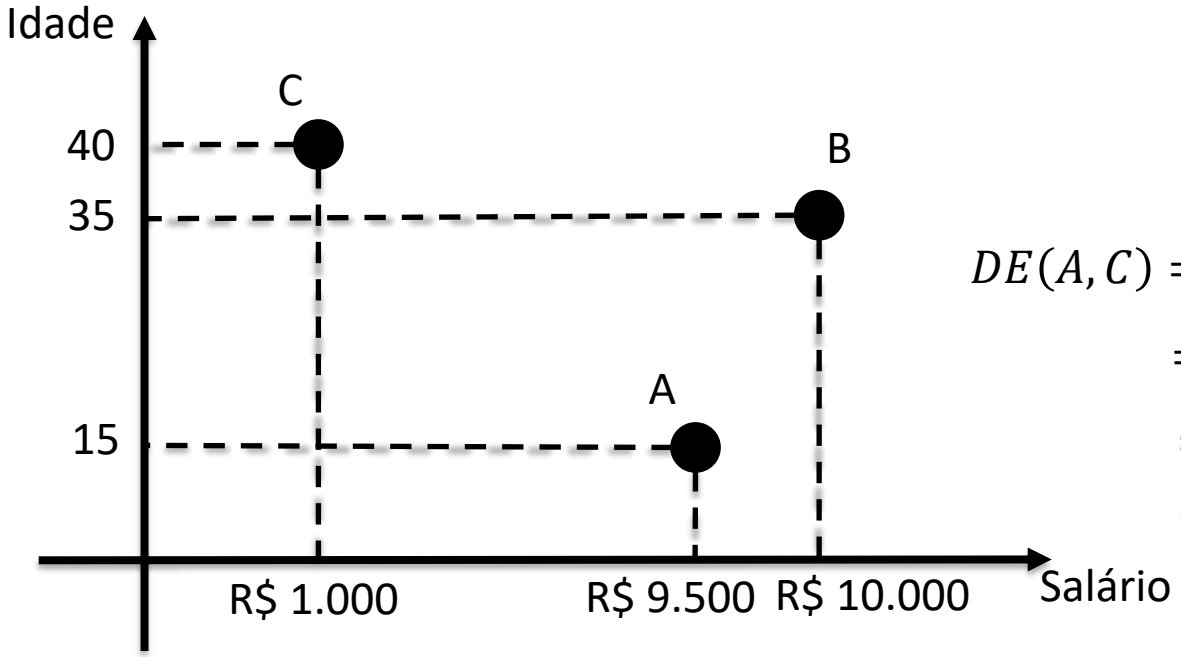
$$DE(A,C) = \sqrt{(15-40)^2+(9500-1000)^2}$$
$$= \sqrt{(25)^2+(8500)^2}$$
$$= \sqrt{625+72250000}$$
$$= 8500{,}04$$

$$DE(A,B) = \sqrt{(15-35)^2+(9500-10000)^2}$$
$$= \sqrt{(20)^2+(-500)^2}$$
$$= \sqrt{400+250000}$$
$$= 500{,}40$$

$$DE(B,C) = \sqrt{(35-40)^2+(10000-1000)^2}$$
$$= \sqrt{(5)^2+(9000)^2}$$
$$= \sqrt{25+81000000}$$
$$= 9000{,}0$$

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000



$$DE(A,C) = \sqrt{(15-40)^2+(9500-1000)^2}$$
$$= \sqrt{(25)^2+(8500)^2}$$
$$= \sqrt{625+72250000}$$
$$= 8500{,}04$$

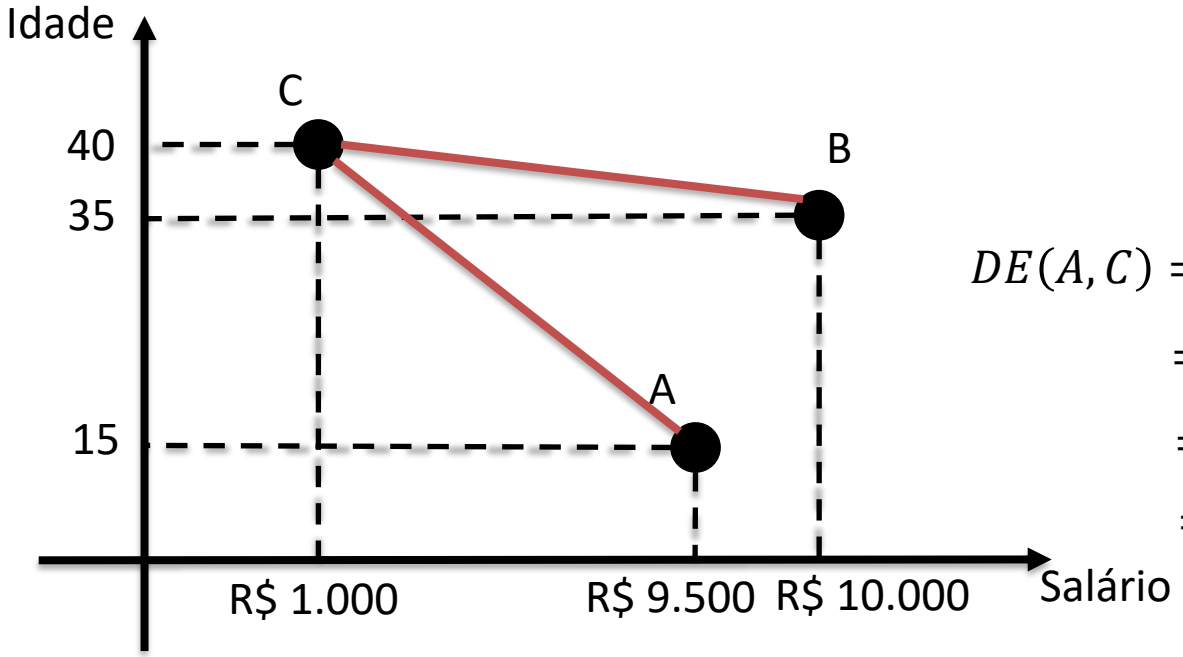
$$DE(A,B) = \sqrt{(15-35)^2+(9500-10000)^2}$$
$$= \sqrt{(20)^2+(-500)^2}$$
$$= \sqrt{400+250000}$$
$$= 500{,}40$$

$$DE(B,C) = \sqrt{(35-40)^2+(10000-1000)^2}$$
$$= \sqrt{(5)^2+(9000)^2}$$
$$= \sqrt{25+81000000}$$
$$= 9000{,}0$$

Qual ponto está mais próximo de C?

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000



$$DE(A,C) = \sqrt{(15-40)^2+(9500-1000)^2}$$
$$= \sqrt{(25)^2+(8500)^2}$$
$$= \sqrt{\mathbf{625} + 72250000}$$
$$= 8500{,}04$$

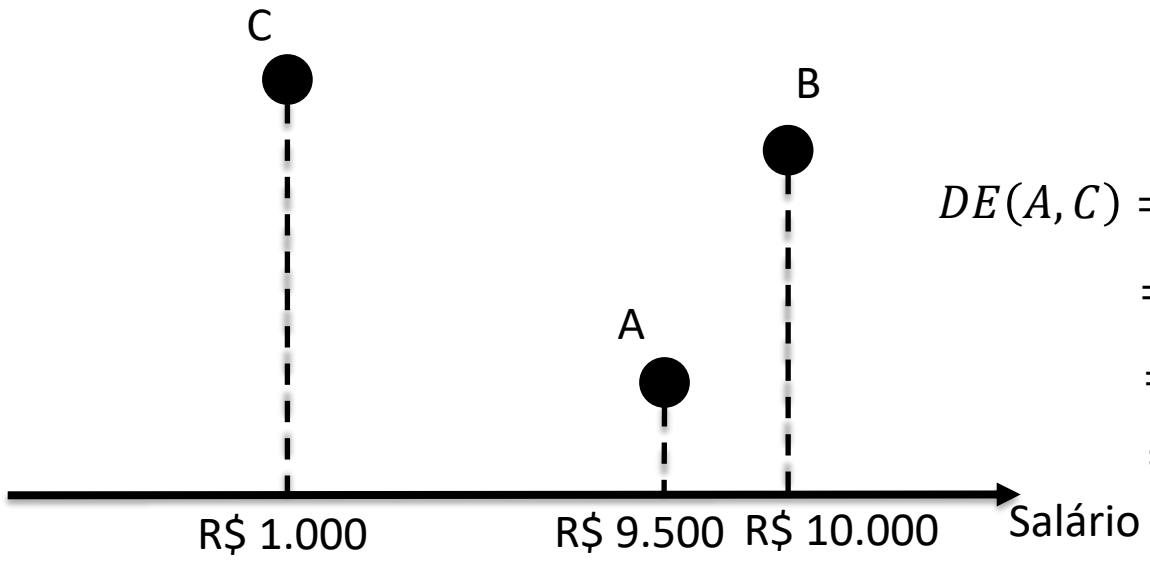
$$DE(A,B) = \sqrt{(15-35)^2+(9500-10000)^2}$$
$$= \sqrt{(20)^2+(-500)^2}$$
$$= \sqrt{\mathbf{400} + 250000}$$
$$= 500{,}40$$

$$DE(B,C) = \sqrt{(35-40)^2+(10000-1000)^2}$$
$$= \sqrt{(5)^2+(9000)^2}$$
$$= \sqrt{\mathbf{25} + 81000000}$$
$$= 9000{,}0$$

O registro A está mais próximo de C que B. A idade quase não faz diferença nas contas.

- **Aplicação em bases reais**
  - Exemplo:

	Salário
A	R$ 9.500
B	R$ 10.000
C	R$ 1.000



$$DE(A,C) = \sqrt{(9500 - 1000)^2}$$
$$= \sqrt{(8500)^2}$$
$$= \sqrt{72250000}$$
$$= 8500$$

$$DE(A,B) = \sqrt{(9500 - 10000)^2}$$
$$= \sqrt{(-500)^2}$$
$$= \sqrt{250000}$$
$$= 500$$

$$DE(B,C) = \sqrt{(10000 - 1000)^2}$$
$$= \sqrt{(9000)^2}$$
$$= \sqrt{81000000}$$
$$= 9000$$

Desconsiderando a idade, o resultado foi quase o mesmo!

K-Nearest Neighbours

- **Aplicação em bases reais**
  - Quando estiver usando qualquer método de aprendizado de máquina baseado no cálculo de **distâncias**, é importantíssimo aplicar uma **padronização** dos dados.
  - Isso fará com que cada característica possua o mesmo **grau de influência** no resultado.

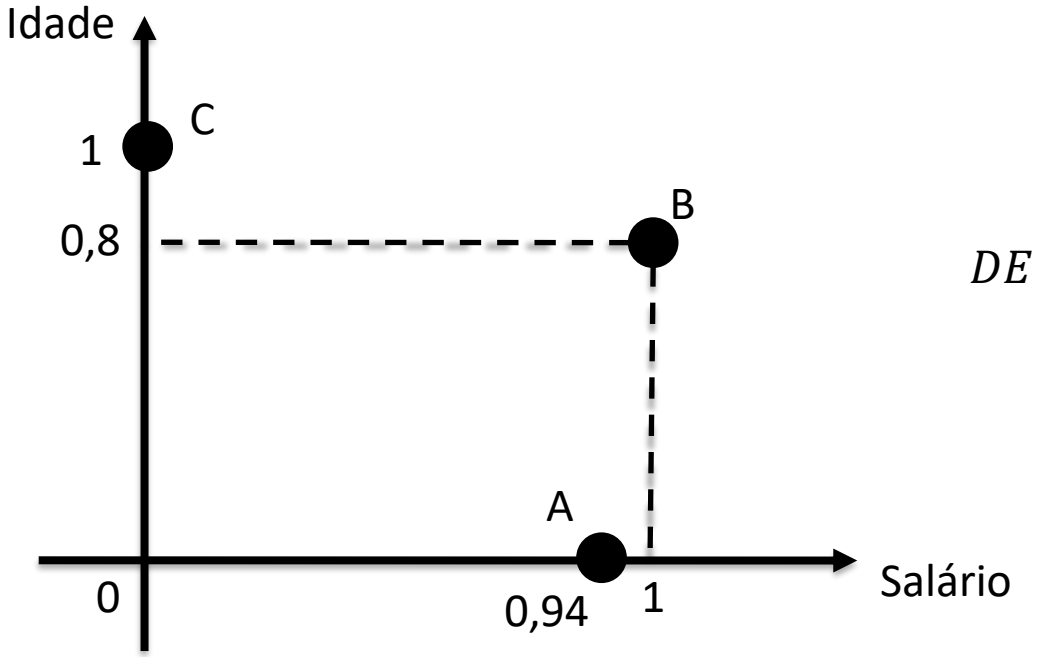
	Idade	Salário
A	15	R$ 9.500
B	35	R$ 10.000
C	40	R$ 1.000

Normalização →

	Idade	Salário
A	0	0,94
B	0,8	1
C	1	0

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	0	0,94
B	0,8	1
C	1	0



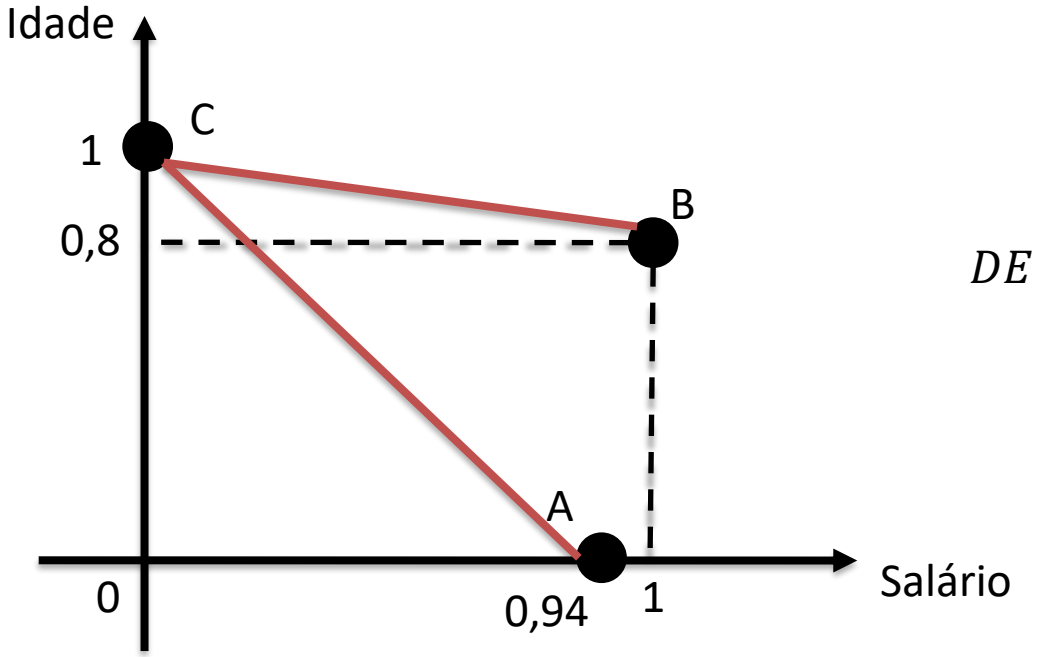
$$DE(A,C) = \sqrt{(0-1)^2+(0{,}94-0)^2}$$
$$= \sqrt{(-1)^2+(0{,}94)^2}$$
$$= \sqrt{1+0{,}8836}$$
$$= 1{,}37$$

$$DE(A,B) = \sqrt{(0-0{,}8)^2+(0{,}94-1)^2}$$
$$= \sqrt{(-0{,}8)^2+(-0{,}06)^2}$$
$$= \sqrt{0{,}64+0{,}0036}$$
$$= 0{,}80$$

$$DE(B,C) = \sqrt{(0{,}8-1)^2+(1-0)^2}$$
$$= \sqrt{(-0{,}2)^2+(1)^2}$$
$$= \sqrt{0{,}04+1}$$
$$= 1{,}02$$

- **Aplicação em bases reais**
  - Exemplo:

	Idade	Salário
A	0	0,94
B	0,8	1
C	1	0



$$DE(A,C) = \sqrt{(0-1)^2+(0{,}94-0)^2}$$
$$= \sqrt{(-1)^2+(0{,}94)^2}$$
$$= \sqrt{1+0{,}8836}$$
$$= 1{,}37$$

$$DE(A,B) = \sqrt{(0-0{,}8)^2+(0{,}94-1)^2}$$
$$= \sqrt{(-0{,}8)^2+(-0{,}06)^2}$$
$$= \sqrt{0{,}64+0{,}0036}$$
$$= 0{,}80$$

$$DE(B,C) = \sqrt{(0{,}8-1)^2+(1-0)^2}$$
$$= \sqrt{(-0{,}2)^2+(1)^2}$$
$$= \sqrt{0{,}04+1}$$
$$= 1{,}02$$

Desta vez, o registro B está mais próximo de C que A.

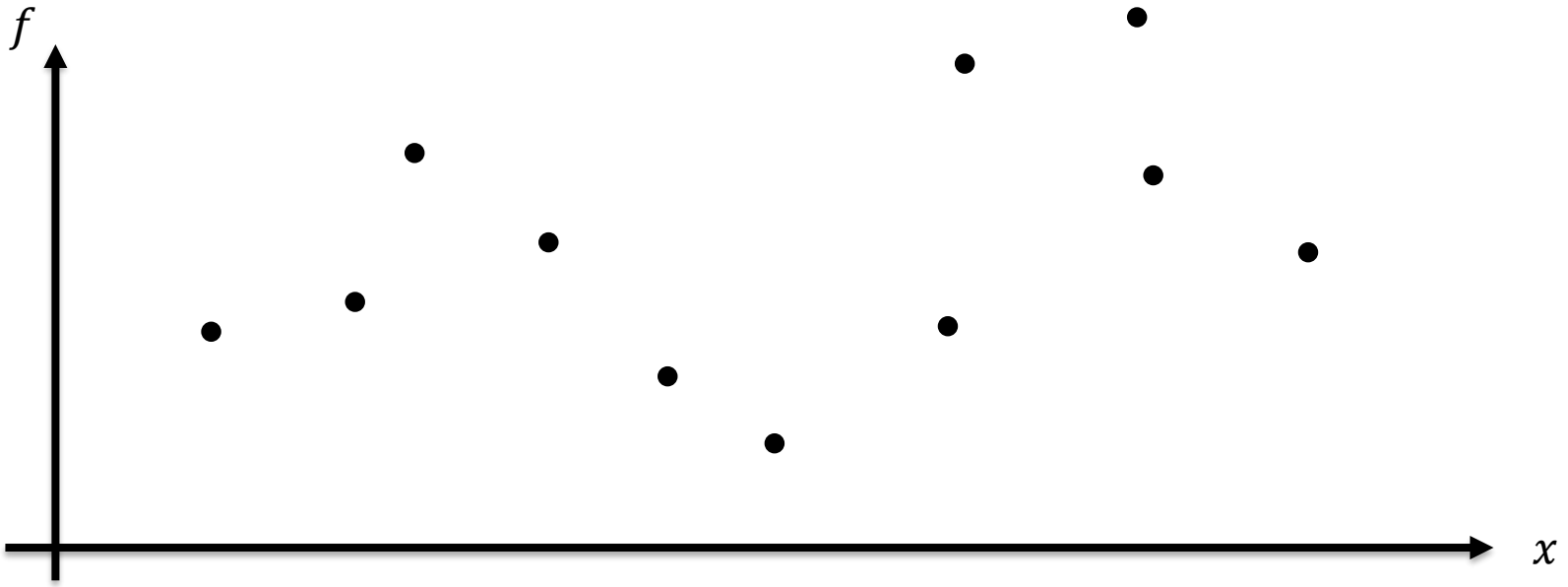
K-Nearest Neighbours

- **Overfitting e underfitting**
  - No **aprendizado supervisionado**, queremos usar os dados de treinamento para construir um modelo que torne possível fazer previsões sobre dados **nunca vistos**, considerando que eles possuem o mesmo comportamento.
  - Se o modelo faz boas previsões em **dados inéditos**, dizemos que que ele foi capaz de **generalizar** as características da base de treinamento.

➢ O nosso objetivo é que essa **generalização** seja a mais precisa possível.

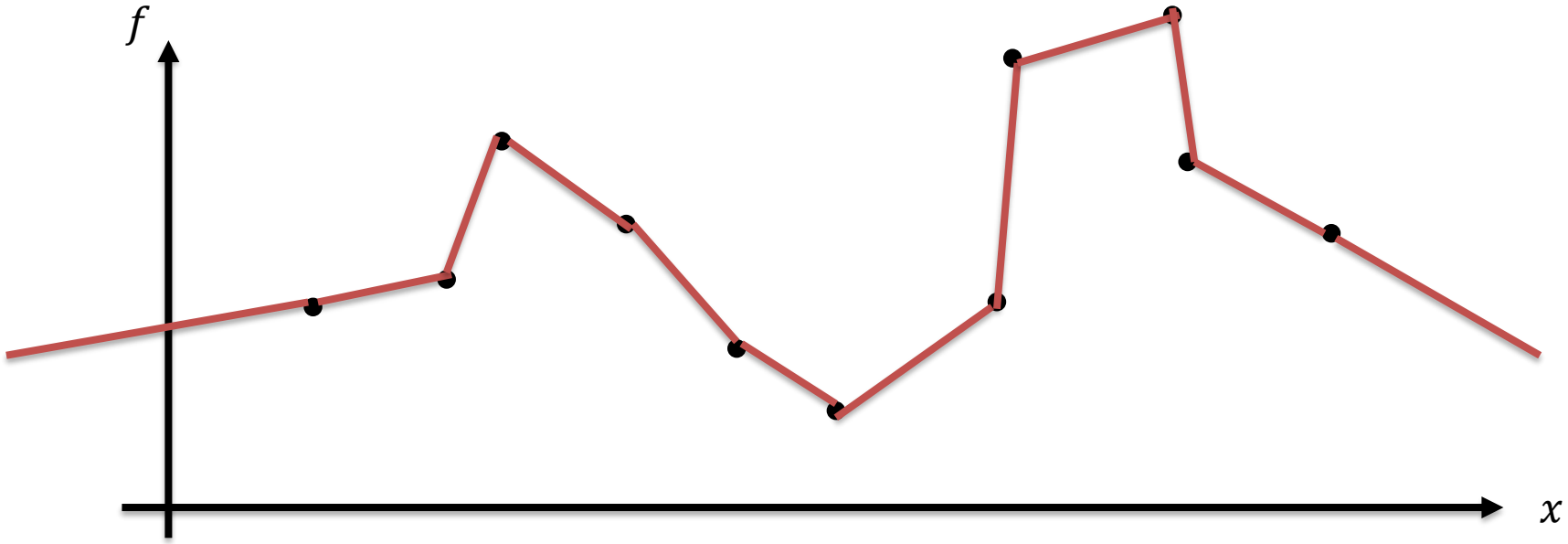
K-Nearest Neighbours

- **Overfitting e underfitting**
  - Quando o modelo tem **excelentes** resultados na **base de treinamento**, mas **péssimos** resultados na **base de testes**, dizemos que ele está “overfitting”.



K-Nearest Neighbours

- **Overfitting e underfitting**
  - Quando o modelo tem **excelentes** resultados na **base de treinamento**, mas **péssimos** resultados na **base de testes**, dizemos que ele está “overfitting”.

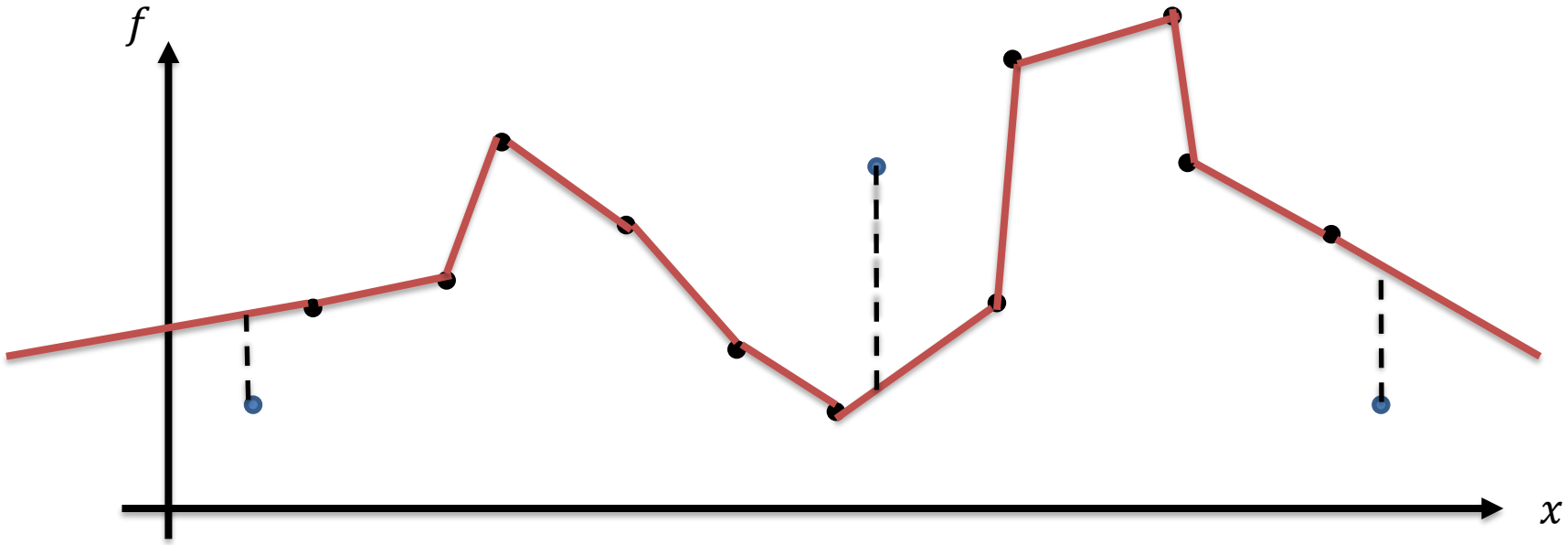


K-Nearest Neighbours

- **Overfitting e underfitting**

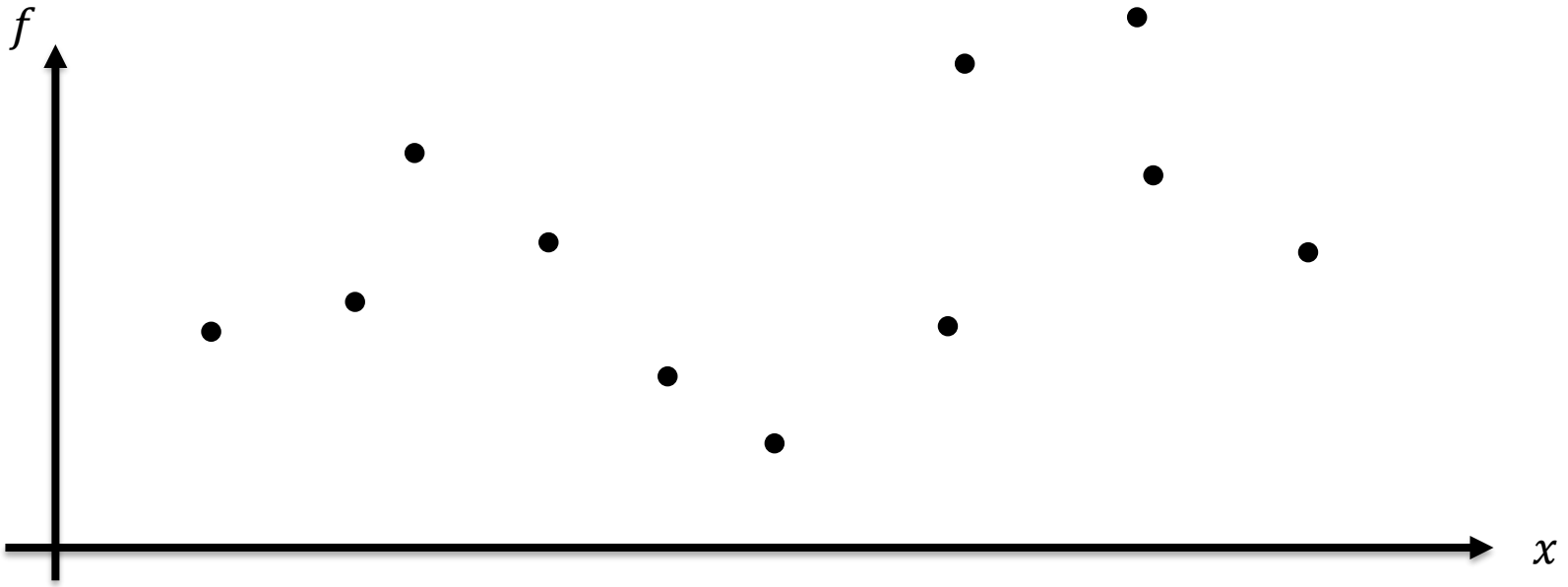
O modelo não é capaz de generalizar!

  – Quando o modelo tem **excelentes** resultados na **base de treinamento**, mas **péssimos** resultados na **base de testes**, dizemos que ele está “overfitting”.



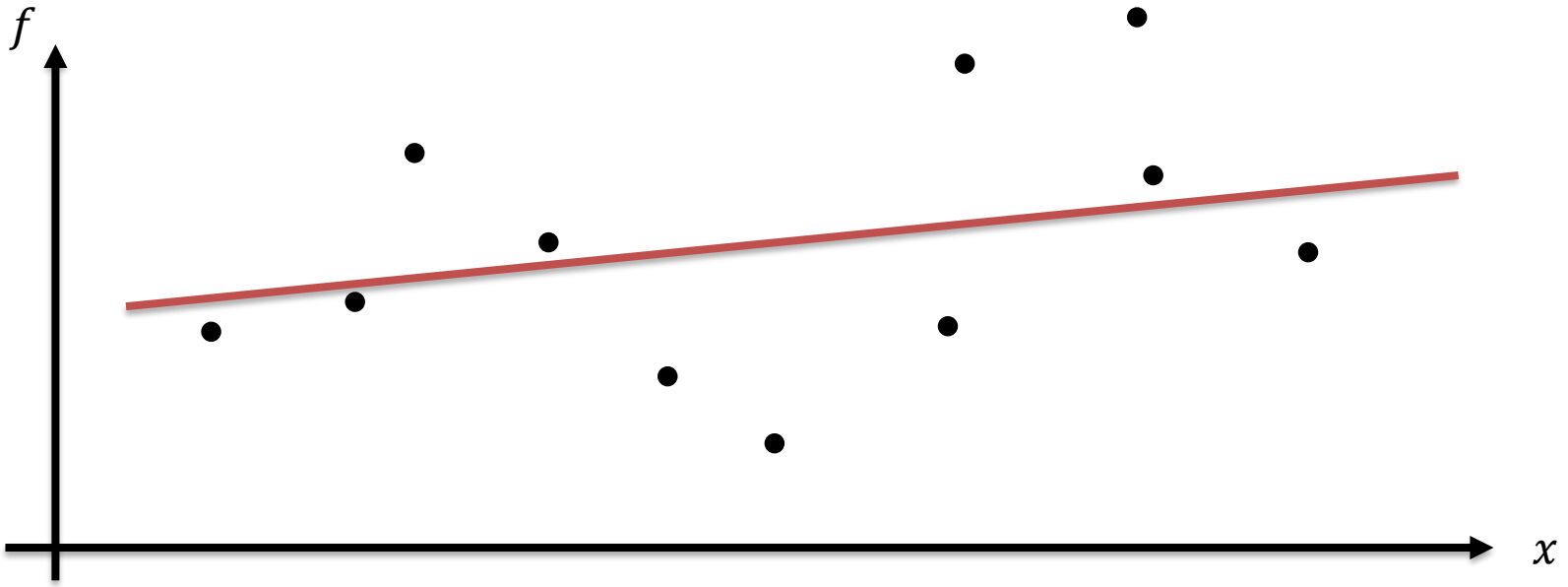
K-Nearest Neighbours

- **Overfitting e underfitting**
  - Se o modelo é **muito simples**, a ponto de **não obter bons resultados** nem mesmo na base de treinamento, dizemos que ele está "underfitting".



K-Nearest Neighbours

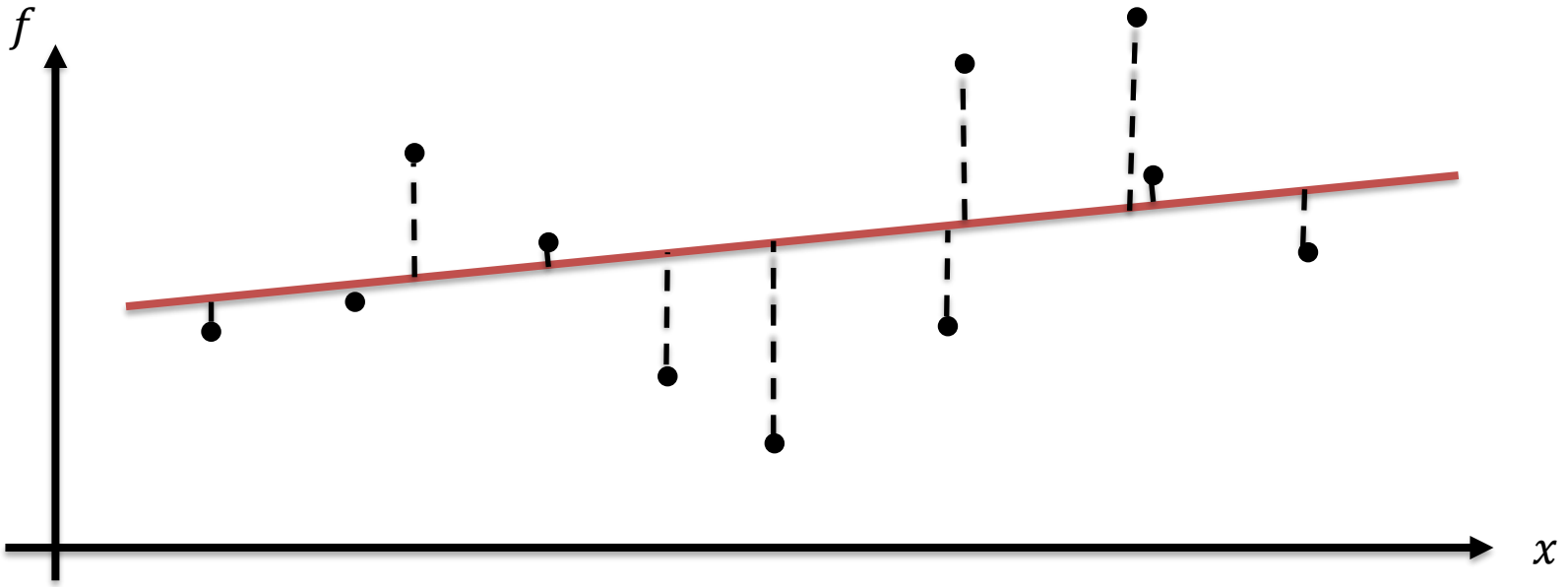
- **Overfitting e underfitting**
  - Se o modelo é **muito simples**, a ponto de **não obter bons resultados** nem mesmo na base de treinamento, dizemos que ele está "underfitting".



K-Nearest Neighbours

O modelo não possui resultados bons na base de treinamento.

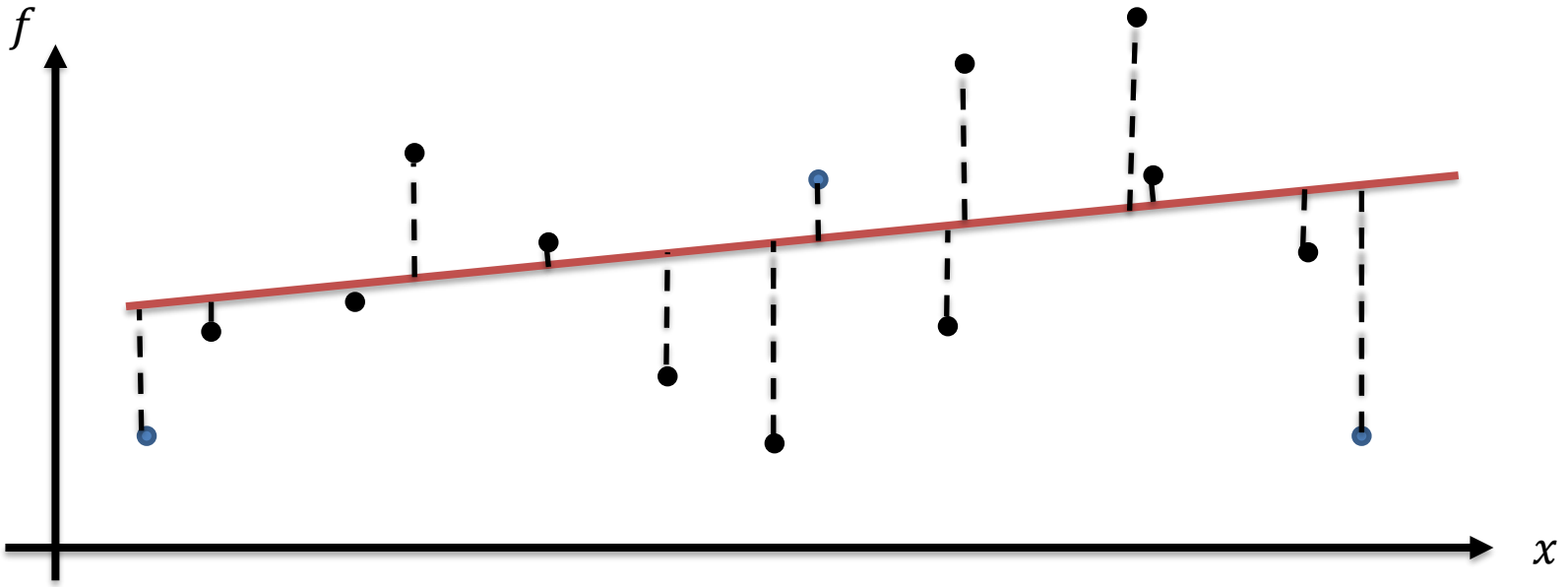
- **Overfitting e underfitting**
  - Se o modelo é **muito simples**, a ponto de **não obter bons resultados** nem mesmo na base de treinamento, dizemos que ele está "underfitting".



K-Nearest Neighbours

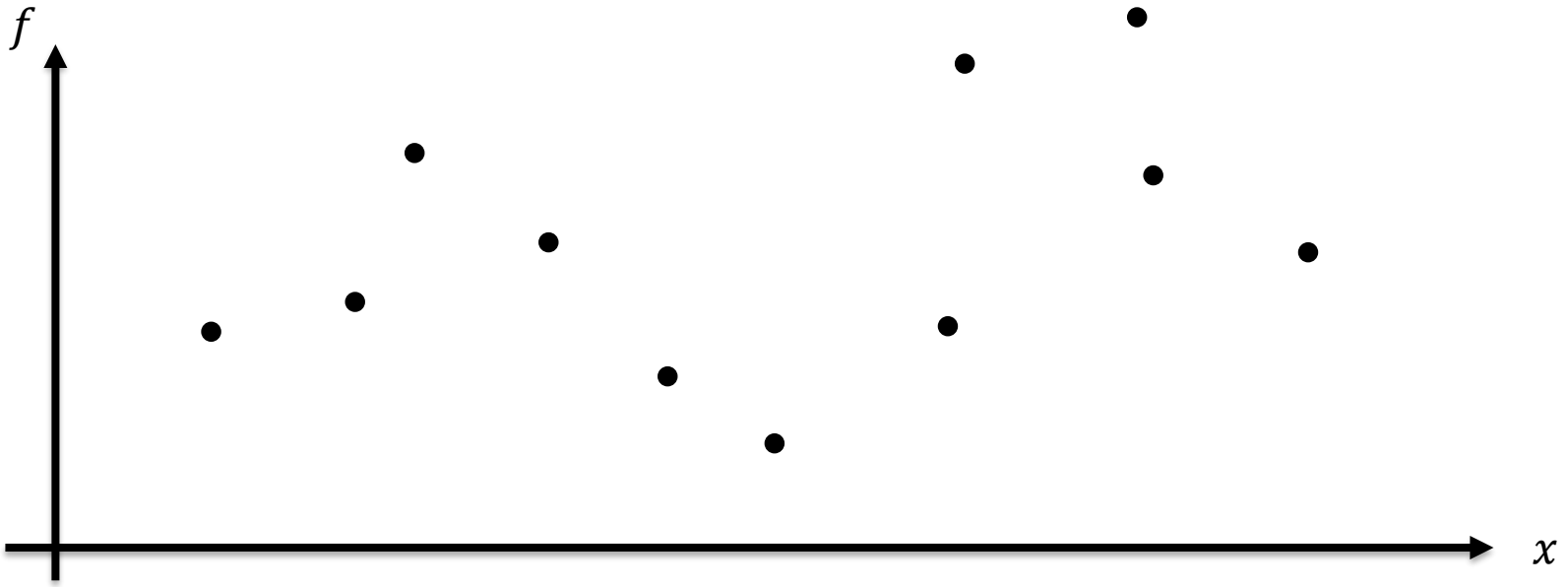
Também não faz boas predições com dados desconhecidos.

- **Overfitting e underfitting**
  - Se o modelo é **muito simples**, a ponto de **não obter bons resultados** nem mesmo na base de treinamento, dizemos que ele está "underfitting".



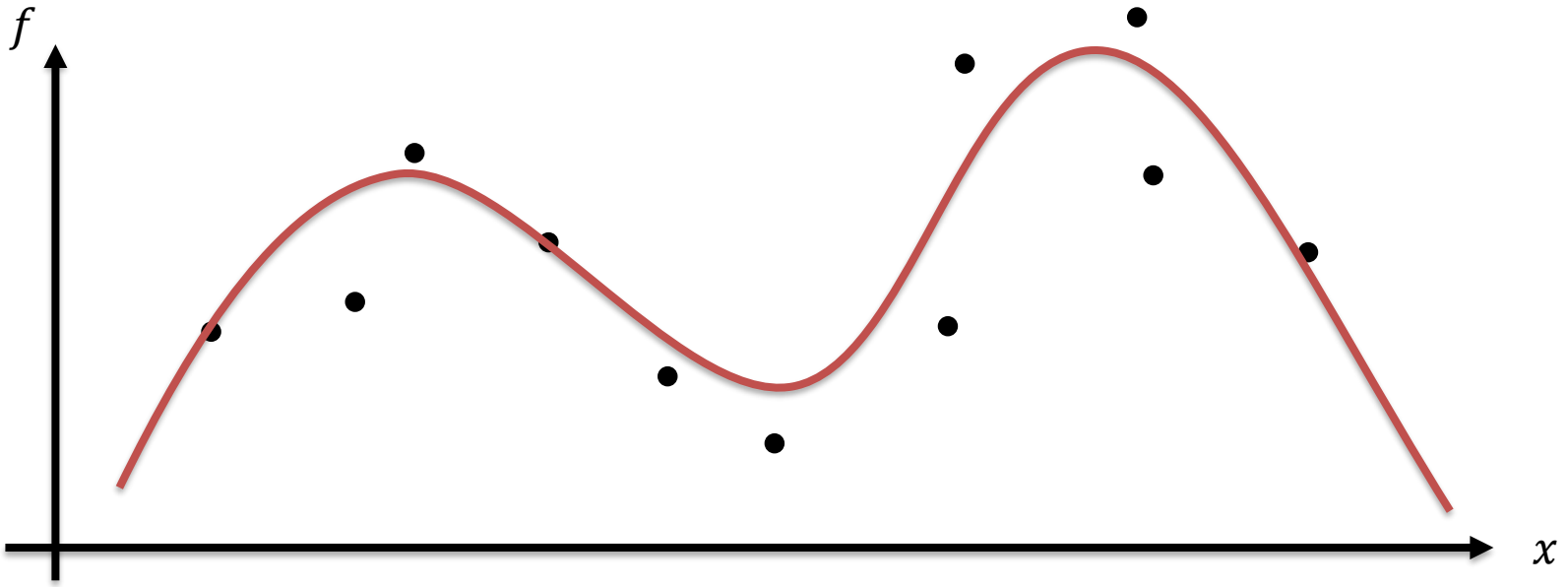
K-Nearest Neighbours

- **Overfitting e underfitting**
  - Um bom modelo é capaz de **generalizar** as características principais dos dados usados no treinamento e trazer **bons resultados** tanto na base de treinamento quanto na de testes!



K-Nearest Neighbours

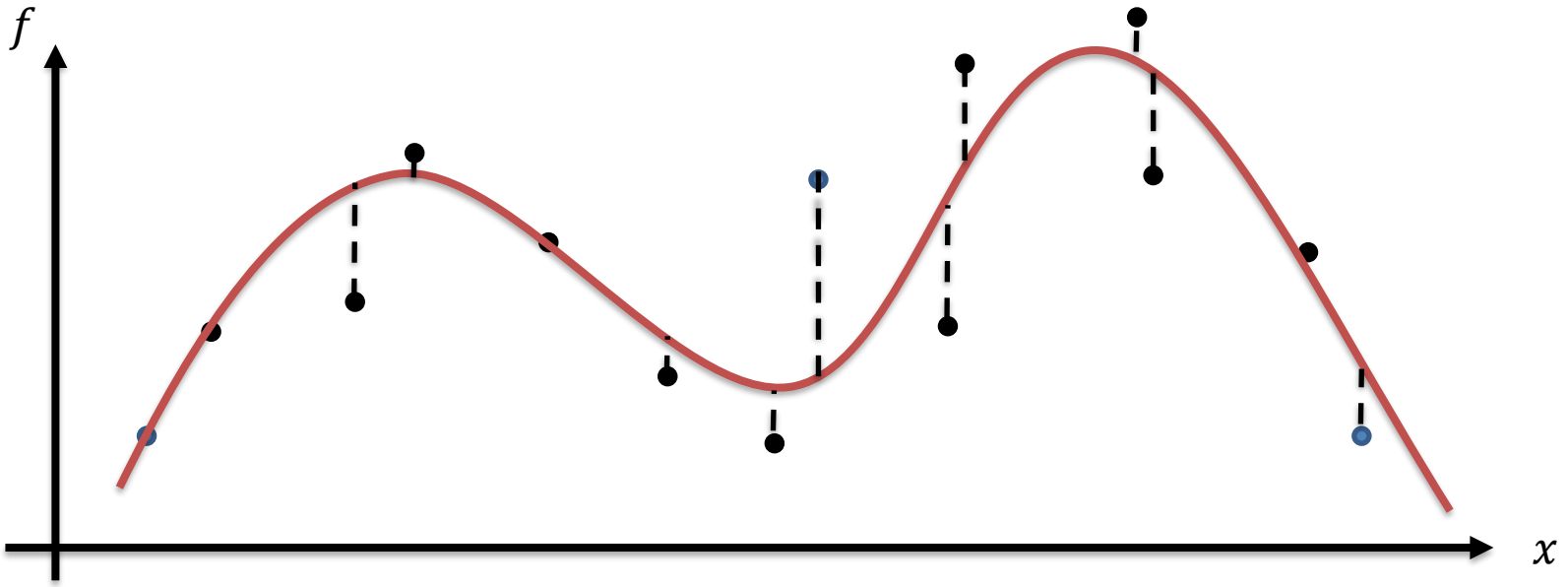
- **Overfitting e underfitting**
  - Um bom modelo é capaz de **generalizar** as características principais dos dados usados no treinamento e trazer **bons resultados** tanto na base de treinamento quanto na de testes!



K-Nearest Neighbours

Não acerta 100%, mas é bom em qualquer conjunto de dados.

- **Overfitting e underfitting**
  - Um bom modelo é capaz de **generalizar** as características principais dos dados usados no treinamento e trazer **bons resultados** tanto na base de treinamento quanto na de testes!



K-Nearest Neighbours

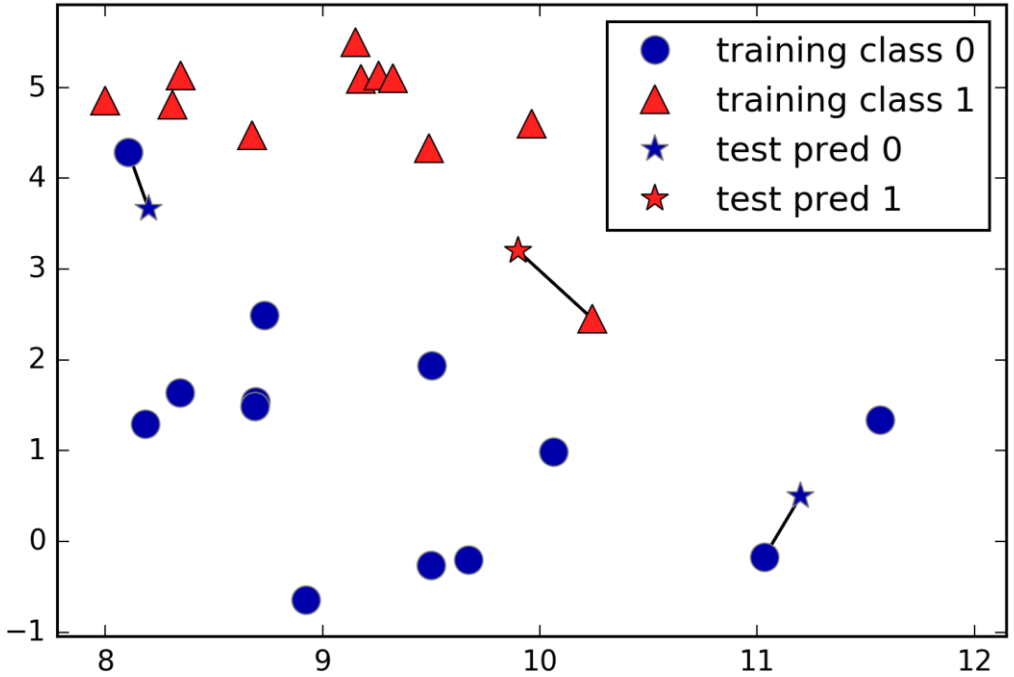
- **Parâmetros de configuração**
  - Cada técnica de aprendizado de máquina possui um conjunto de **parâmetros de configuração**.
  - Dependendo da escolha destes parâmetros, o algoritmo pode apresentar **melhores** ou **piores** resultados.
  - Os principais parâmetros de configuração do KNN são:
    - **Valor de K**: número de vizinhos a serem considerados.
    - **Métrica de distância**: fórmula usada para calcular a similaridade entre os registros.

Não existe um valor padrão que funciona para **qualquer caso**: sempre é necessário testar!

K-Nearest Neighbours

- **Escolha de K**
  - A escolha do valor de K poderá causar um **overfitting** ou um **underfitting** no problema.
  - Exemplo:

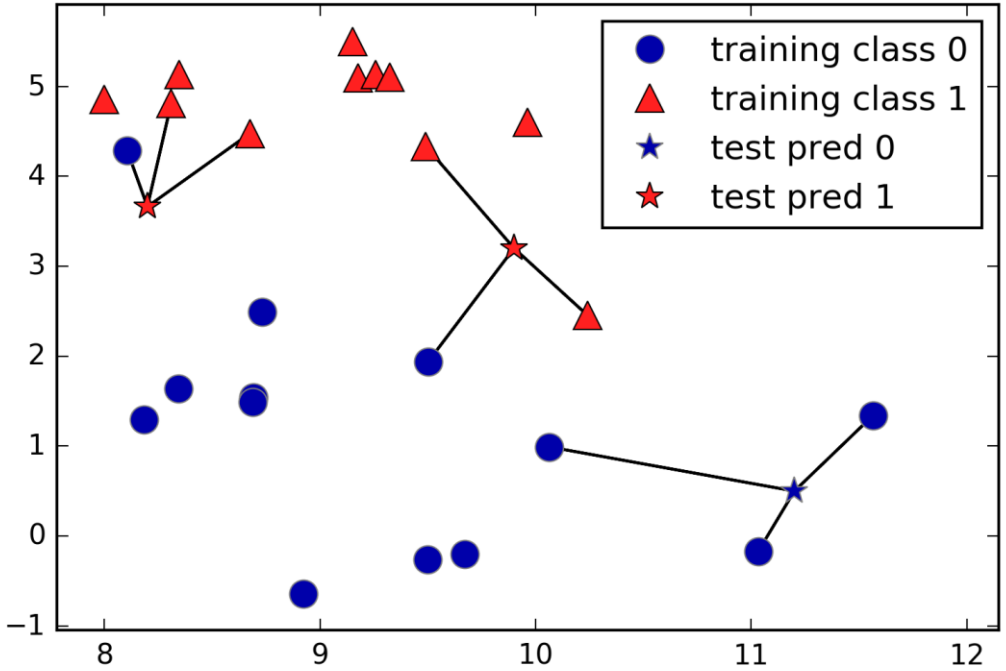
K=1



K-Nearest Neighbours

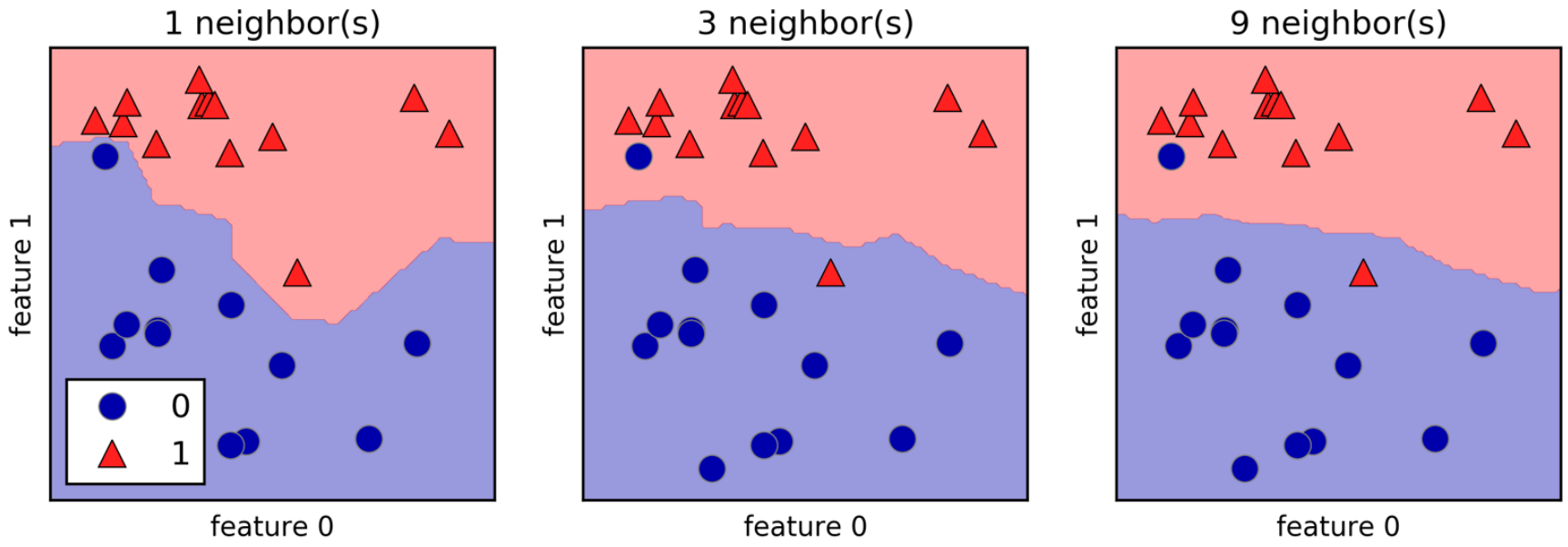
- **Escolha de K**
  - A escolha do valor de K poderá causar um **overfitting** ou um **underfitting** no problema.
  - Exemplo:

K=3



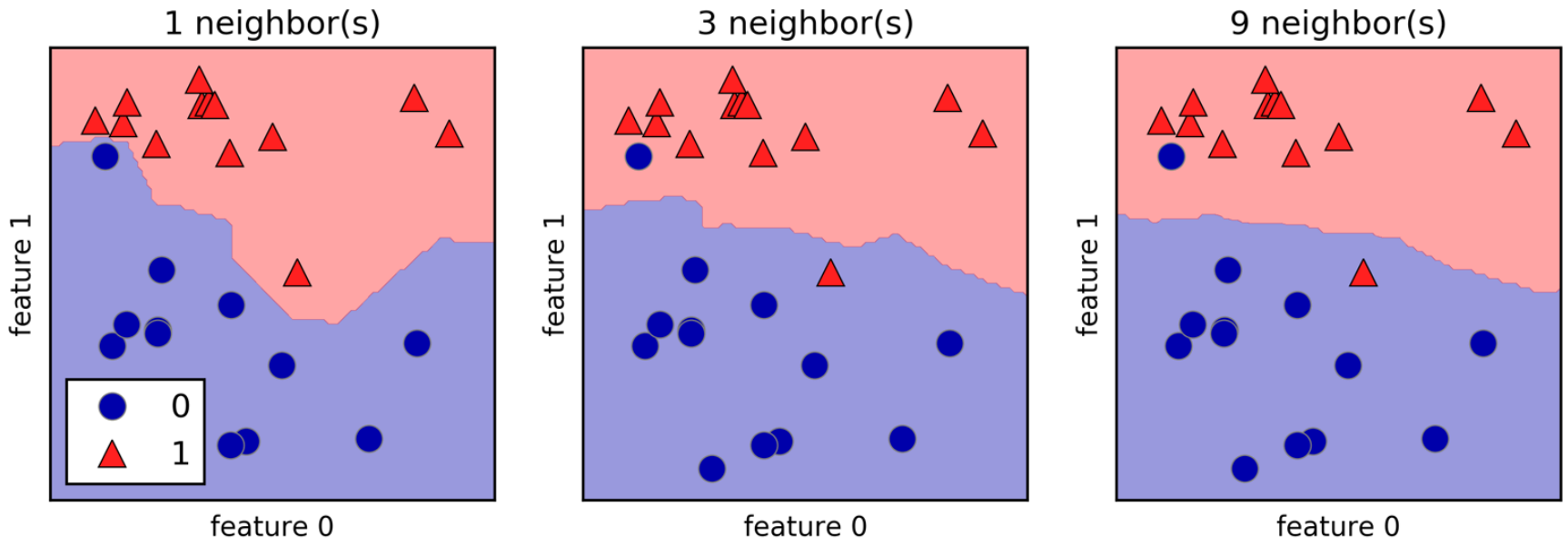
K-Nearest Neighbours

- **Escolha de K**
  - A **fronteira** de decisão fica mais **suave** a medida que o valor de **K aumenta**.



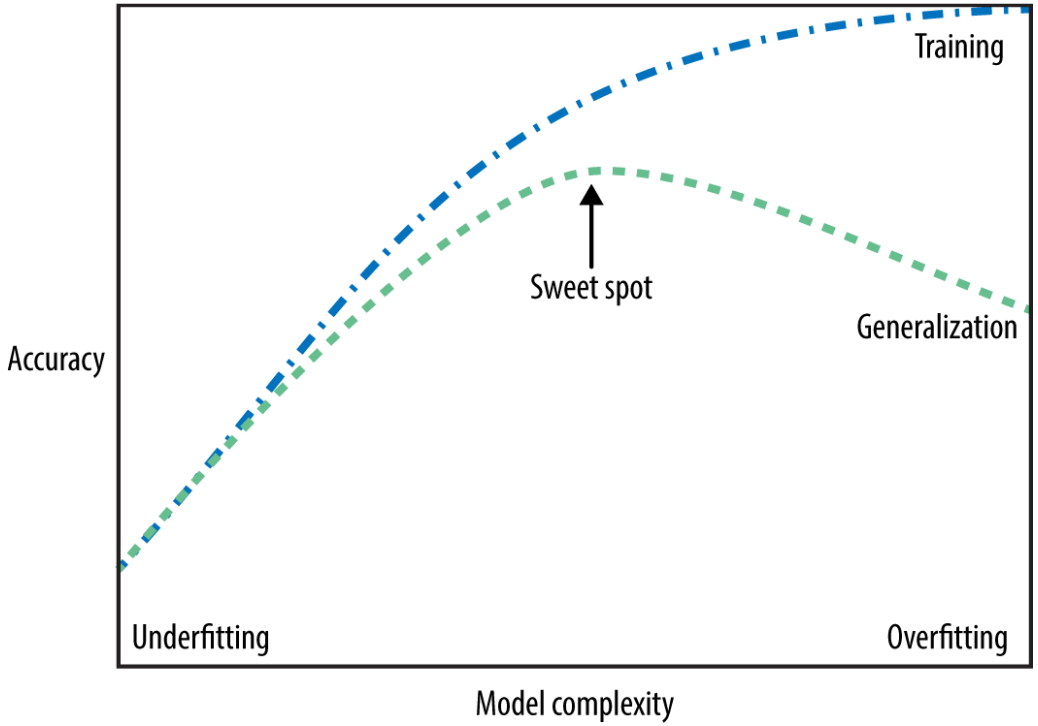
K-Nearest Neighbours

- **Escolha de K**
  - Isso indica que o modelo está ficando **mais simples**: risco de **underfitting**!



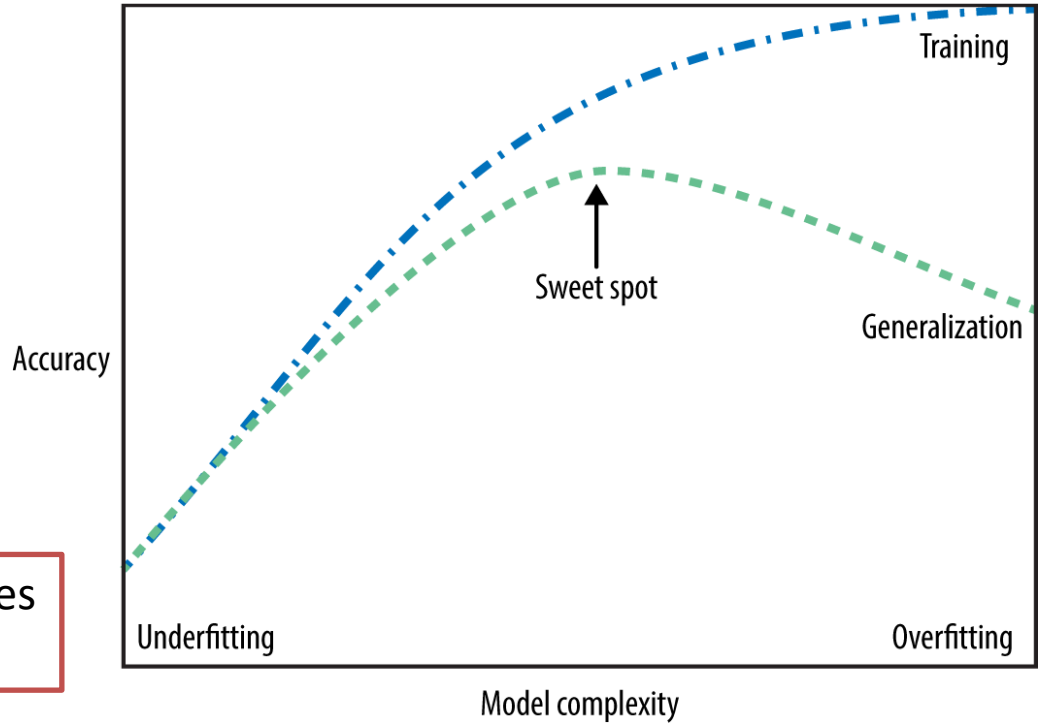
K-Nearest Neighbours

- **Escolha de K**
  - Em geral, deve-se ajustar os parâmetros do algoritmo levando em conta o seguinte gráfico:



K-Nearest Neighbours

- **Escolha de K**
  - Em geral, deve-se ajustar os parâmetros do algoritmo levando em conta o seguinte gráfico:



Modelo excessivamente simples
K alto

Modelo excessivamente complexo
K baixo

- **Construindo o script**

  – Primeiramente, o script de pré-processamento deve ser executado, selecionando quais as transformações precisam ser feitas na base.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\PythonWorkspace\2025.1\aula4

...tual\2025-1_Técnicas de Aprendizado de Máquina (reformulação)\aula 3\Programas\pre-processamento.py

pre-processamento.py × credit_data.csv ×

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 31 18:32:59 2025

@author: marco
"""

import pandas as pd
base = pd.read_csv('credit_data.csv')
resumo = base.describe()


# ==========================================================================
#                        Tratando valores inválidos
# ==========================================================================

# Verifica idades inválidas
base.loc[base['Age'] < 0]

# Apagar a coluna
# base.drop('Age', axis=1, inplace=True)

# Apagar apenas os registros com problema
# base.drop(base[base['Age'] < 0].index, inplace=True)

# Preencher os valores com a média
idade_media = base['Age'][base['Age'] > 0].mean().round()
base.loc[base['Age'] < 0, 'Age'] = idade_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, …
checking_maioria	str	6	little
classe	DataFrame	(600, 1)	Column names: Risk
classe_teste	DataFrame	(150, 1)	Column names: Risk
classe_treinamento	DataFrame	(450, 1)	Column names: Risk
cols_classe	list	1	['Risk']
cols_previsores	Index	(18,)	Index: 18 entries, Age …

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024,
15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]:
   ...: """
   ...: Created on Mon Mar 31 18:32:59 2025
   ...:
   ...: @author: marco
   ...: """
   ...:
   ...: import pandas as pd
   ...: base = pd.read_csv('credit_data.csv')
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 3, Col 36 UTF-8 CRLF RW Mem 75%

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificador = KNeighborsClassifier(n_neighbors=[?] metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)
```

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificador = KNeighborsClassifier(n_neighbors=?, metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise                                   gem de acertos
from skle                                   ion_matrix, acc
acuracia_                                   asse_teste, pre
matriz_te                                   asse_teste, previsoes)
```

Nesta linha, definimos os parâmetros de configuração do método.

Esta distância é equivalente à Euclidiana.

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificador = KNeighborsClassifier(n_neighbors=?, metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
y_score(classe_teste, previsoes)
_matrix(classe_teste, previsoes)
```

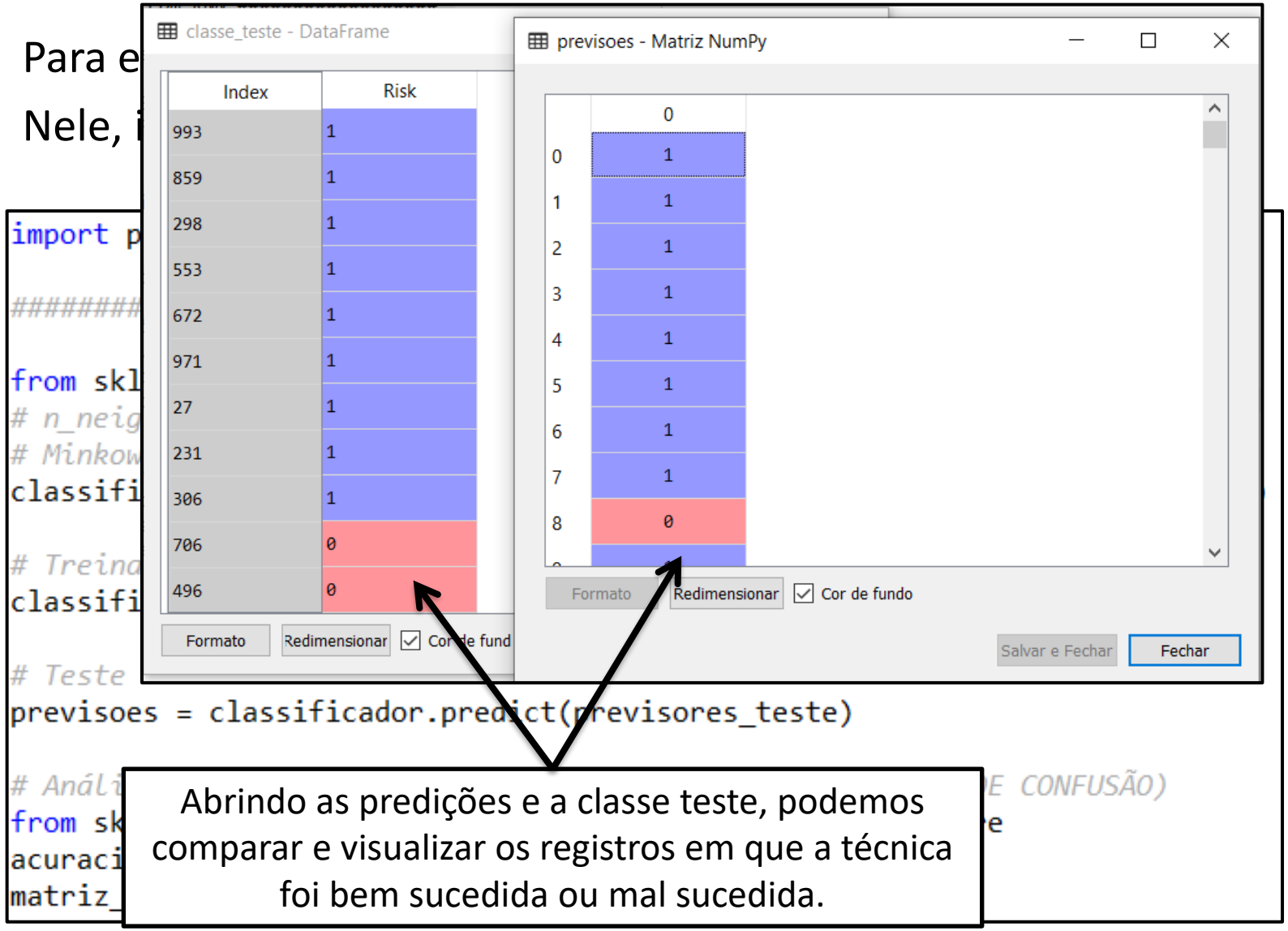
Chamando “fit”, o algoritmo irá aprender com os dados de treinamento.

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificador = KNeighborsClassifier(n_neighbors=[?] metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = ac                                    es)
matriz_teste = conf                                    es)
```

O predict irá retornar as previsões feitas para os valores inéditos de teste, usando o modelo treinado.

- **Construindo o script**
  - Para e
  - Nele, i



Abrindo as predições e a classe teste, podemos comparar e visualizar os registros em que a técnica foi bem sucedida ou mal sucedida.

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski c                                    iana
classificador                                    _neighbors= ? metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)
```

O accuracy score retorna o percentual de acertos do método.

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com KNN ##################

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificado                    n_neighbors= ?  metric='minkowski', p=2)

# Treinament
classificado                    nto, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)
```

A matriz de confusão irá nos revelar quantos foram os falso-positivos e falso-negativos.

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN.py”.
  - Nele, insira o seguinte código:

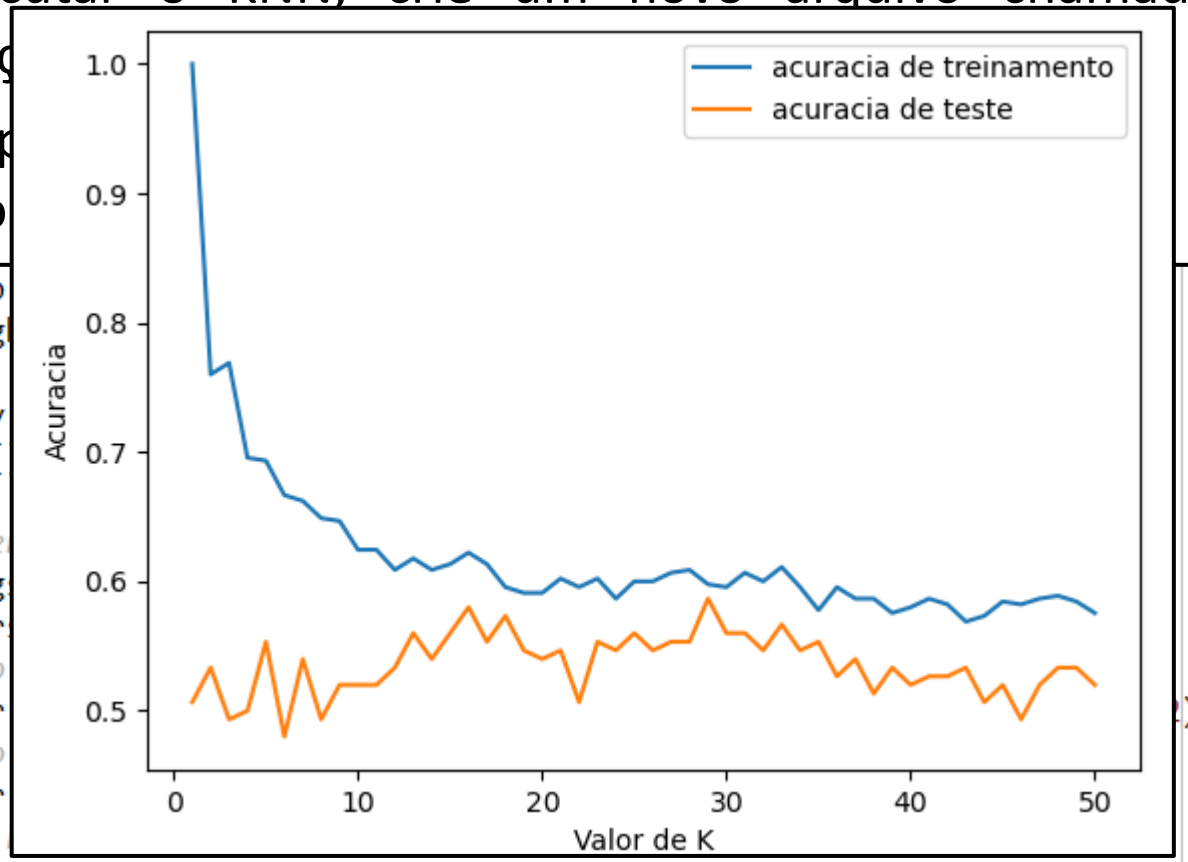
Como descobrir o melhor valor de K para cada caso?

```
import pandas as pd

################## Classificação com KNN ##

from sklearn.neighbors import KNeighborsClassifier
# n_neighbors é o valor de K
# Minkowski com p=2 é Distância Euclidiana
classificador = KNeighborsClassifier(n_neighbors=?, metric='minkowski', p=2)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)
```

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN-configuração.py”.
  - Este script servirá para testarmos diferentes valores de K e avaliarmos qual seria o valor ideal para este problema.

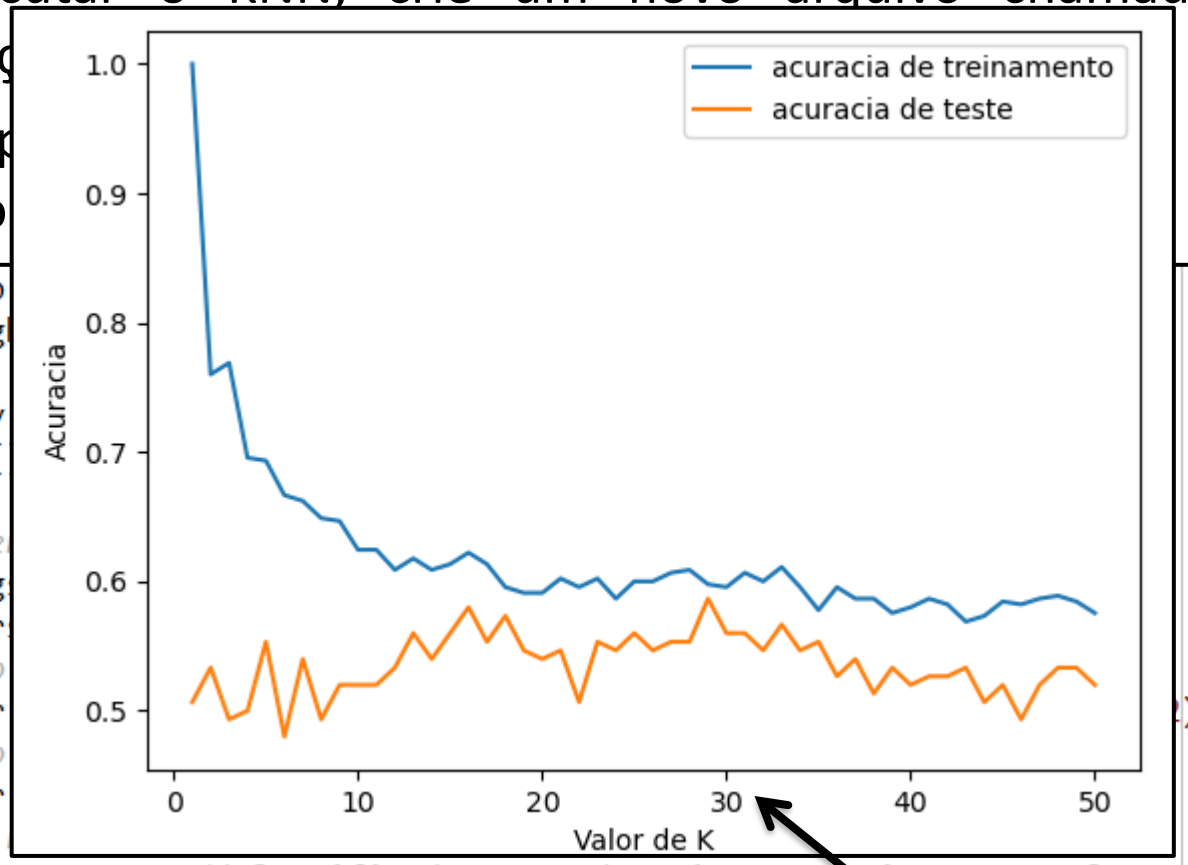
```
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

training_accuracy = []
test_accuracy = []

# tentando diferentes valores de K: de 1 to 50
neighbors_settings = range(1, 51)
for k in neighbors_settings:
    # Construindo o modelo
    classificador = KNeighborsClassifier(n_neighbors=k, metric='minkowski', p=2)
    # Treinamento do modelo
    classificador.fit(previsores_treinamento, classe_treinamento)
    # Gravando o resultado para os dados de treinamento
    training_accuracy.append(classificador.score(previsores_treinamento, classe_treinamento))
    # Gravando o resultado para os dados de teste (generalização)
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))
plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend()
```

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN-configurac
  - Este scrip de K e avaliarmo




```
import matplotlib
from sklearn.neigl

training_accuracy
test_accuracy = [

# tentando difere
neighbors_setting
for k in neighbor
    # Construindo
    classificador
    # Treinamento
    classificador
    # Gravando o
    training_accuracy.append(classificador.score(previsores_treinamento, classe_treinamento))
    # Gravando o resultado para os dados de teste (generalização)
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))
plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend()
```

- **Construindo o script**
  - Para executar o KNN, crie um novo arquivo chamado “KNN-configuraç
  - Este scrip de K e avaliarmo




```
import matplotlib
from sklearn.neigl

training_accuracy
test_accuracy = [

# tentando difere
neighbors_setting
for k in neighbor
    # Construindo
    classificador
    # Treinamento
    classificador
    # Gravando o
    training_accuracy.append(classificador.score(previsores_treinamento, classe_treinamento))
    # Gravando o resultado para os dados de teste (generalização)
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))
plt.plot(neighbors_settings, training_accuracy, label=
plt.plot(neighbors_settings, test_accuracy, label="tes
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend()
```

Neste caso, um K=29 parece ser a melhor decisão.

- **Construindo o script**
  - Voltando no script do KNN, podemos colocar o valor de K e coletar os resultados.

```
############## Classificação com KNN ##########

import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

# minkowski com p=2 é a distância euclidiana
classificador = KNeighborsClassifier(n_neighbors=29, metric='minkowski', p=2)

#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)
```

- **Construindo o script**
  - Voltando no script do KNN, podemos colocar o valor de K e coletar os resultados.

```
############## Classificação com KNN ##########

import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

# mi
clas                                                    ', p=2)
#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, acc
acuracia_teste = accuracy_score(classe_teste,prev
matriz_teste = confusion_matrix(classe_teste,previsões)
```

acuracia_teste	float	1	0.5866666666666667
matriz_teste	Array of int64	(2, 2)	[[43 27] [35 45]]

Precisamos registrar este resultado para futuras comparações!

- **Registrando os resultados**
  - Devem ser feitos **diversos testes e execuções** da técnica, variando os parâmetros de configuração e os diferentes tratamentos de dados.
  - Além disso, também pode ser interessante alterar a **escolha de variáveis previsoras** utilizadas.

Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão
Classe Majoritária	-	-	0.4667	70 0 80 0
KNN	K=5 Distancia Euclidiana	Criação de variáveis dummy	0.5533	41 29 38 42
KNN	K=29 Distancia Euclidiana	Criação de variáveis dummy	0.5867	43 27 35 45
KNN	K=30 Distancia Euclidiana	Criação de variáveis dummy Padronização	0.6333	49 21 34 46

FIM